



Storitveno usmerjene arhitekture

izr. prof. dr. Boštjan Šumak

1

Nazadnje

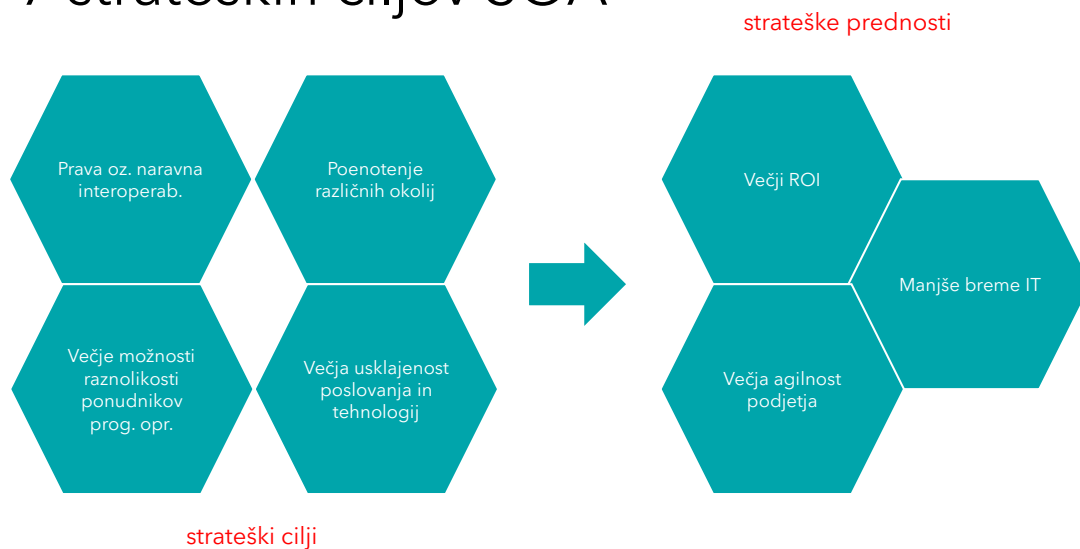
- Primer razvoja storitve TrafficService na osnovi JavaScript
- Interoperabilnost storitev (WS-I, profili)
- Odjemalci storitev (Java, .NET, JavaScript)
- Storitveno vodilo (Enterprise Service Bus)
- Življenjski cikli razvoja SOA
 - BOTTOM-UP pristop
 - TOP-DOWN pristop
 - AGILNI pristop
- Storitveno naravnana analiza
- Modeli storitev (entitetna, poslovna, podporna)
- Tipi storitvenih arhitektur
 - arhitektura storitve, arhitektura kompozicije, arhitektura inventarja, storitveno naravnana poslovna arhitektura

2

Strateški cilji SOA

3

7 strateških ciljev SOA



Prvi štirje strateški cilji omogočajo doseganje zadnjih treh ciljev oz. prednosti!

4

1. Prava oz. naravna interoperabilnost

- Cilj SOAe: vzpostavitev **naravne (vgrajene) interoperabilnosti** med storitvami, tako da:
 - se bistveno zmanjša potreba po naknadnih integracijskih projektih,
 - so storitve zasnovane tako, da so **kompatibilne** in **interoperabilne** ne glede na platformo, tehnologijo ali izvajalca,
 - storitve komunicirajo preko **jasno definiranih, standardiziranih vmesnikov**.
- Posledica:
 - pojem »integracija sistemov« **postopoma izgublja na pomenu**,
 - če je storitvena usmerjenost dosledno uporabljena na vseh področjih podjetja, **postane prava interoperabilnost med storitvami pravilo in ne izjema**.

Interoperabilnost = sposobnost programa/aplikacije/storitve interakcije in izmenjave podatkov



Integracija = trud, ki je potreben za doseganje interoperabilnosti med programi- Integracija je nujna povsod, kjer programi niso kompatibilni oz. interoperabilni

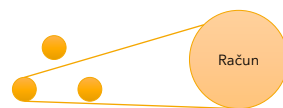
5

1. Prava oz. naravna interoperabilnost

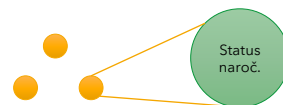
- Storitve, ki jih razvijeta projektni ekipi A in B, so med seboj interoperabilne.
- Projektna ekipa C jih lahko neposredno ponovno uporabi pri razvoju nove rešitve.
- Za povezovanje storitev ni potrebno izvajati dodatnih integracijskih projektov, temveč zgolj konfiguracija in kompozicija obstoječih storitev.



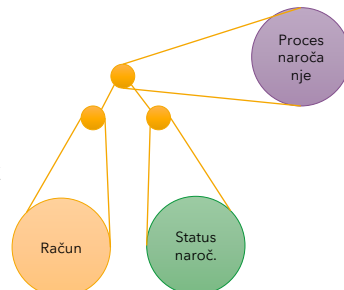
Projektna ekipa A



Projektna ekipa B



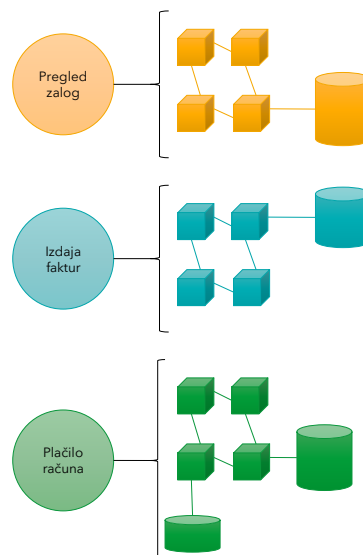
Projektna ekipa C



6

2. Poenotenje različnih okolij

- Cilj SOA: **poenotenje raznolikih aplikacijskih in tehnoloških okolij v podjetju**,
 - vzpostavitev **zveznega ("federiranega") okolja** preko standardiziranih storitev.
- Kako to dosežemo:
 - s **standardiziranimi storitvami in enotnimi pogodbenimi vmesniki** (service contracts),
 - s storitvami, ki so grajene tako, da so **primerne za sestavljanje (kompozicijo)** v kompleksnejše procese in rešitve,
 - storitve so na voljo **v različnih domenah** in organizacijskih enotah podjetja, a delujejo kot del ene povezane storitvene krajine.



7

3. Več možnosti za raznolikost ponudnikov programske opreme

- Cilj SOA:
 - omogočiti **preoblikovanje, razširitev in nadgradnjo obstoječih rešitev z novimi tehnologijami in produkti različnih ponudnikov**,
 - **zmanjšati odvisnost od posameznega dobavitelja** (vendor lock-in).
- Kako to dosežemo:
 - z uvedbo **ustrezne ravni abstrakcije** pri načrtovanju storitev,
 - z zasnovo SOA, ki je **neodvisna od konkretnih produktov** posameznih ponudnikov,
 - z **jasno definiranimi storitvami na poslovni ravni**, ki jih je mogoče implementirati z različnimi tehnologijami.
- Raznolikost sicer ni vedno prioriteta - je pa ključno, da arhitektura omogoča izbiro in zamenljivost ponudnikov, ko poslovne potrebe to zahtevajo.

8

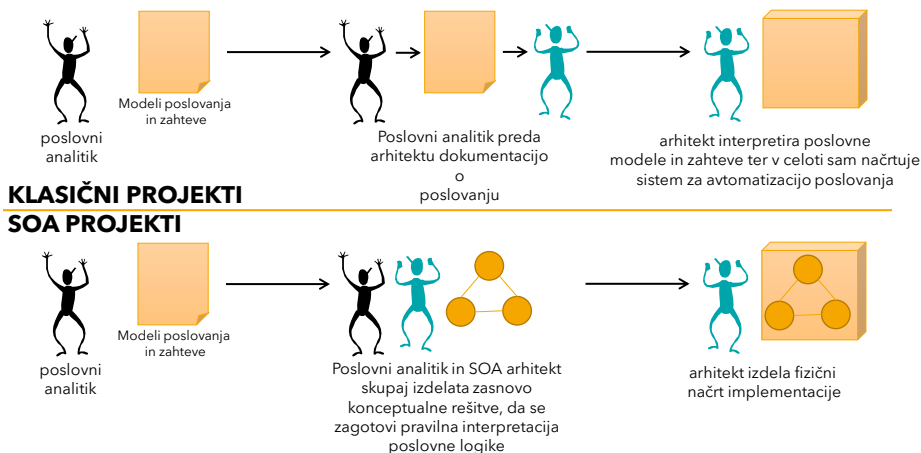
4. Večja usklajenost poslovanja in tehnologij

- *Usklajenost poslovanja in tehnologij=*
»stopnja, do katere informacijski sistemi in IT rešitve podpirajo ter se razvijajo skladno s potrebami poslovanja.«
- SOA prispeva k večji usklajenosti tako, da:
 - poslovne potrebe prevajamo v storitve, ki odražajo poslovne funkcije in procese,
 - spremembe v poslovanju vodijo v usklajene spremembe v storitvah,
 - IT ne razvija rešitev izolirano, temveč v neposredni povezavi s poslovnimi cilji.

9

Večja usklajenost poslovanja in tehnologij

- Ključno:
 - domenski (poslovni) eksperti in IT arhitekti tesno sodelujejo pri analizi, modeliranju in načrtovanju storitev,
 - poslovne zmožnosti (business capabilities) se prevedejo v storitve,
 - s tem se zmanjša razkorak med poslovnimi pričakovanji in dejansko IT-rešitvijo (»business-IT alignment«).



10

5. Večji ROI

- ROI (return of investment) =

"vrednost oziroma prihranek, dosežen z uvedbo rešitve, v primerjavi s stroški njene implementacije in upravljanja"

- SOA in storitveno naravnano načrtovanje prispevata k večjemu ROI, ker:
 - omogočata razvoj **agnostičnih storitev** (neodvisnih, interoperabilnih in ponovno uporabnih),
 - spodbujata **ponovno uporabo** obstoječih storitev pri novih rešitvah,
 - **zmanjšujeta obseg in kompleksnost integracijskih** projektov,
 - olajšata uvajanje **sprememb in nadgradenj z manjšimi stroški in krajšimi roki**.

11

6. Večja agilnost podjetja

- Agilnost =

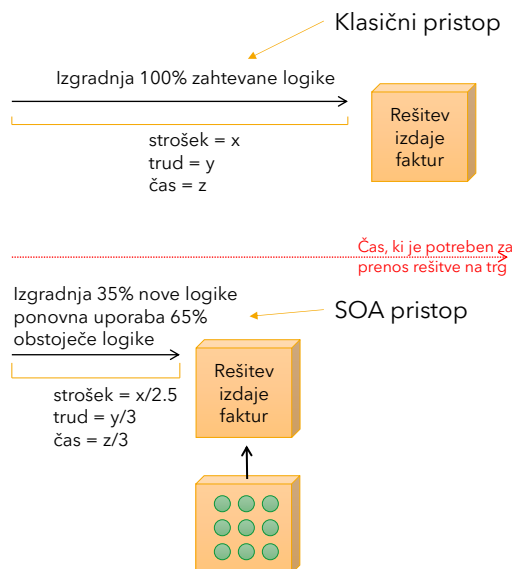
"sposobnost hitrega, učinkovitega in zmogljivega prilagajanja na spremembe"

- SOA podpira agilnost podjetja, ker:
 - nove zahteve rešujemo s kompozicijo obstoječih storitev,
 - spremembe pogosto zahtevajo prilagoditev posameznih storitev, ne pa celotnih monolitnih sistemov,
 - se skrajša čas od ideje do delujoče rešitve (time-to-market).

12

Večja agilnost podjetja

- Ko ima podjetje na voljo zbirko zveznih, interoperabilnih in agnostičnih storitev:
 - se **čas in trud za izgradnjo novih rešitev ali spremembo obstoječih občutno zmanjšata**,
 - projektne ekipe se lahko osredotočijo na **dodano vrednost in poslovne funkcionalnosti**,
 - uporaba standardnih storitev **zmanjša tveganja pri uvajanju sprememb**.



13

7. Manjše breme za IT

- Konsistentna izgradnja storitveno naravnanih rešitev prinaša za IT:
 - **zmanjšanje stroškov zaradi »odpada«** (rešitev, ki jih je treba hitro zavreči ali temeljito predelati),
 - **manj razvoja redundantnih**, podvojenih rešitev,
 - **nižje operativne stroške** zaradi enotnejšega in preglednejšega okolja,
 - **zmanjšanje stroškov, povezanih z upravljanjem in nadzorom** storitev (SOA governance) ter z njihovim nadaljnjim razvojem in evolucijo,
 - **prehod** iz vloge »gasilske službe« **v vlogo načrtnega skrbnika storitvene krajine**.

14

SOA Manifesto

Ali Arsanjani
Grady Booch
Toufic Boubez
Paul C. Brown
David Chappell
John deVadoss

Thomas Erl
Nicolai Josuttis
Dirk Krafzig
Mark Little
Brian Loesgen
Anne Thomas Manes

Joe McKendrick
Steve Ross-Talbot
Stefan Tilkov
Clemens Utschig-
Utschig
Herbjörn Wilhelmsen

15

SOA manifest

- SOA Manifesto – **manifest za storitveno usmerjeno arhitekturo**
 - dokument, ki **opisuje temeljna načela, vrednote in cilje storitveno usmerjene arhitekture** (SOA),
 - nastal kot odgovor na potrebo po bolj strukturiranem in usklajenem pristopu k uporabi SOA,
 - presega zgolj tehnične vidike ter poudarja tudi organizacijske in poslovne koristi.
- Manifest opredeljuje ključne vrednote in načela, ki naj bi jih organizacije upoštevale pri uvajanju SOA, da bi z arhitekturo učinkovito podprle svoje strateške cilje.

16

SOA manifest - vrednote

- Manifesto poudarja, da so nekatere vrednote pomembnejše od drugih:

poslovna vrednost pred tehnološkimi strategijami

- Poslovne zahteve so gonilo razvoja; tehnologije služijo kot sredstvo za njihovo uresničevanje.
- SOA je namenjena reševanju poslovnih izzivov in ustvarjanju vrednosti za organizacijo, ne zgolj uvajanju novih tehnologij.

strateški cilji pred koristmi posameznega projekta

- Storitve predstavljajo poslovne zmogljivosti, ki so na voljo celotnemu podjetju, ne le enemu projektu.
- Pomembnejše od kratkoročnih projektnih koristi so strategije, ki omogočajo dolgoročno prilagodljivost in trajnostno arhitekturo.

notranja interoperabilnost pred integracijami po meri

- Storitvena usmerjenost vodi k storitvam, ki so po naravi interoperabilne.
- Namesto dragih, enkratnih integracijskih rešitev stremimo k vnaprej načrtovani, »vgrajeni« interoperabilnosti.

17

SOA manifest - vrednote

- Manifesto poudarja, da so nekatere vrednote pomembnejše od drugih:

storitve v skupni rabi pred storitvami za posebne namene

- Storitve enkapsulirajo logiko, ki je ponovno uporabna v različnih kontekstih.
- Skupna raba storitev povečuje agilnost, zmanjšuje podvajanje in olajša vzdrževanje.

prilagodljivost pred lokalno optimizacijo

- Optimizacija je še vedno pomembna, kjer je smiselna, vendar ne sme onemogočati prilagodljivosti.
- Prednost imajo arhitekturne rešitve, ki omogočajo enostavno kompozicijo storitev in agilni razvoj novih poslovnih zmogljivosti.

evolucijsko izpopolnjevanje pred popolno začetno rešitvijo

- Poslovno okolje se nenehno spreminja, zato morajo biti storitve zasnovane za enostavno nadgradnjo.
- Bolje je imeti arhitekturo, ki jo lahko postopno izboljšujemo, kot iskati idealno, a toga začetno rešitev.

18

SOA manifest - načela

Spoštuj družbeno strukturo in strukturo moči organizacije

- Uvajanje SOA mora upoštevati obstoječe organizacijske odnose in jih postopno prilagajati, ne pa rušiti.

Sprejetje SOA naj bo usmerjeno v poslovanje, ne v tehnologijo

- Pobuda za SOA mora izhajati iz poslovnih potreb in ciljev, tehnologija je podporni dejavnik.

Zavedajte se, da SOA zahteva spremembe na številnih ravneh

- Spremenijo se načini vodenja projektov, življenjski cikel razvoja programske opreme, organizacija ekip ipd.

Obseg sprejetja SOA je lahko različen - trud naj bo obvladljiv

- SOA lahko uvajamo postopno, na izbranih področjih; pomembno je, da je obseg realen in obvladljiv.

Upravljanje SOA (SOA governance) je ključno na vsakem nivoju sprejetja

- Potrebni so dogovorjeni standardi, pravila in odgovornosti za razvoj, uporabo in nadgradnjo storitev.

Začnite z majhnimi, izvedljivimi koraki

- Prve uspešne pilotne rešitve pomagajo graditi zaupanje in razumevanje koristi SOA.

SOA ni produkt - ne moremo je kupiti

- Produkti in standardi sami po sebi ne zagotavljajo SOA.

Storitvena usmerjenost je neodvisna od tehnologije in ponudnika

- SOA lahko uresničimo z različnimi tehnologijami in standardi; ključna je paradigma, ne platforma.

Vzpostavite enoten nabor poslovnih standardov in politik

- Ti naj temeljijo na industrijskih standardih, de facto standardih, standardih

19

SOA manifest - načela

Prizadevajte si za enotnost navzven, dopuščajte raznolikost navznoter.

- Navzven (na ravni storitev in vmesnikov) naj bo okolje standardizirano; interno so lahko uporabljene različne tehnologije in programskih modeli.

Storitve identificirajte v sodelovanju s podjetjem in deležniki.

- Pri zasnovi SOA vključite ključne poslovne deležnike, domenske eksperte in IT - storitve naj odražajo poslovne zmožnosti.

Povečajte uporabo storitev z upoštevanjem trenutnega in prihodnjega obsega uporabe.

- Storitve načrtujte za večkratno ponovno uporabo, spremembe in širšo uporabo v prihodnosti.

Preverjajte, ali storitve resnično izpolnjujejo poslovne zahteve in cilje.

- Uspešnost storitev merite glede na poslovne rezultate, ne le tehnične kazalnike.

Razvijajte storitve na podlagi dejanske uporabe.

- Načrtujte iterativno: izkušnje iz dejanske uporabe naj vodijo v izboljšave in nadaljnji razvoj storitev.

20

SOA manifest - pomen

Skupna vizija za SOA

- Manifest ponuja enotno vizijo in smernice za organizacije, ki uvajajo SOA.
- Pomaga zmanjšati nesporazume in nerealna pričakovanja v zvezi s to arhitekturo.

Usmeritev na poslovne koristi

- Poudarek je na poslovnih vrednotah, prilagodljivosti in dolgoročni podpori poslovnim ciljem, ne le na tehnoloških vidikih.

Spodbujanje prilagodljivosti in vzdržnosti

- SOA je opisana kot arhitektura, ki omogoča hitro prilagajanje, obvladovanje kompleksnosti in ohranjanje konkurenčnosti na dinamičnem trgu.

Večja ponovna uporabnost storitev

- Manifest poudarja pomen ponovne uporabe in neodvisnosti storitev.
- To organizacijam omogoča znižanje stroškov razvoja, enostavnejše vzdrževanje ter hitrejši razvoj novih rešitev.

21

Načrtovalski vzorci SOA

22

SOA načrtovalski vzorci

Načrtovalski vzorci storitvenega inventarja

Načrtovalski vzorci storitev

Načrtovalski vzorci kompozicij storitev

Načrtovalski vzorci na osnovi REST

Sestavljeni vzorci



23

Struktura opisa vzorcev

Zahteva oz. izhodišče

Problem

Rešitev

Aplikacija

Vplivi

Povezava s principi storitvene naravnosti

SOA arhitekturni stil

Povezani vzorci

Povezava s storitveno-usmerjenimi cilji

24

1. Načrtovalski vzorci storitvenega inventarja

Inventar storitev je zbirka neodvisno standardiziranih in upravljanih storitev

Temeljni vzorci
inventarja

Vzorci logičnih
nivojev
inventarja

Vzorci
centralizacije
inventarja

Vzorci
implementacije
inventarja

Vzorci
upravljanja
inventarja

25

Temeljni vzorci inventarja

Osnovni vzorci načrtovanja arhitekture inventarja storitev

Kanonični protokol
(Canonical Protocol)

• Kako oblikovati storitve, da se izognemo potrebi po preslikavi protokolov?

Kanonična shema
(Canonical Schema)

• Kako oblikovati storitve, da se izognemo potrebi po preslikavi podatkovnih modelov?

Domenski inventar storitev
(Domain Inventory)

• Kako je mogoče zagotoviti čim večjo stopnjo ponovne uporabnosti storitev skozi kompozicijo, ko standardizacija za celotno podjetje ni mogoča?

Poslovni inventar storitev
(Enterprise Inventory)

• Kako je mogoče zagotoviti storitve za čim večjo ponovno uporabo v kompoziciji?

Centralizacija logike
(Logic Centralization)

• Kako se izogniti uporabi logike podvojenih storitev?

Storitveni nivoji
(Service Layers)

• Kako organizirati storitve v inventarju na osnovi funkcionalne podobnosti?

Storitvena normalizacija
(Service Normalization)

• Kako se izogniti podvajanju logike storitev na nivoju inventarja storitev?

26

Vzorci logičnih nivojev inventarja

Najbolj pogoste vrste nivojev storitev, na osnovi katerih lahko storitve v inventarju storitev združimo v logične plasti

Abstrakcija entitete (Entity Abstraction)

- *Kako je mogoče agnostično poslovno logiko ločiti, ponovno uporabiti in neodvisno upravljati?*

Abstrakcija procesa (Process Abstraction)

- *Kako je mogoče ne-agnostično poslovno logiko ločiti in neodvisno upravljati?*

Abstrakcija podpornih storitev (Utility Abstraction)

- *Kako je mogoče skupno ne-poslovno logiko ločiti, ponovno uporabiti in upravljati neodvisno?*

Abstrakcija mikro-opravlil (Micro Task Abstraction)

- *Kako je mogoče neagnostično logiko s posebnimi zahtevami za obdelavo ločiti in upravljati neodvisno?*

27

Vzorci centralizacije inventarja

Nabor vzorcev za centralizacijo ključnih delov arhitekture inventarja storitev, ki temeljijo na nadgradnji temeljnih arhitekturnih vzorcev.

Centralizacija politik (Policy Centralization)

- *Kako lahko politike normaliziramo in jih dosledno izvajamo žez več storitev?*

Centralizacija procesa (Process Centralization)

- *Kako je mogoče centralno upravljati logiko abstraktnega poslovnega procesa?*

Centralizacija pravil (Rules Centralization)

- *Kako se lahko poslovna pravila abstrahirajo in upravljajo centralno?*

Centralizacija sheme (Schema Centralization)

- *Kako je mogoče oblikovati pogodbe storitev, da se izognemo podvajanju podatkov?*

28

Vzorci implementacije inventarja

Različni vidiki načrtovanja implementacije in možnosti za izgradnjo arhitekture inventarja storitev.

Kanonični viri (Canonical Resources)	• Kako se je mogoče izogniti nepotrebnemu nesorazmerju infrastrukturnih virov?
Med-domenski nivo uporabe (Cross-domain Utility Layer)	• Kako se je mogoče izogniti redundantni logiki v inventarjih domenskih storitev?
Dualni protokoli (Dual Protocols)	• Kako lahko inventar storitev preseže svoje omejitve kanoničnega protokola in hkrati ostane standardiziran?
Končna točka inventarja (Inventory Endpoint)	• Kako je mogoče inventar storitev zaščititi pred dostopom od zunaj in hkrati zagotoviti dostop do storitev zunanjim odjemalcem?
Storitvena mreža (Service Grid)	• Kako je mogoče podatke o odloženem stanju storitve prilagoditi in ohraniti odpornost na napake?
Repozitorij stanj (State Repository)	• Kako se lahko podatki o stanju storitve ohranijo dlje časa, ne da bi porabili vire izvajalnega časa storitve?
Storitve s stanjem (Stateful Services)	• Kako je mogoče ohraniti in upravljati podatke o stanju storitve brez porabe virov v času izvajanja storitve?
Nadgrajeni protokoli (Augmented Protocols)	• Kako lahko inventar storitev nadgradi svojo ponudbo storitev brez spremembe svojega kanoničnega protokola ali ga ogrozil?

29

Vzorci upravljanja inventarja

Načrtovalski vzorci povezani z upravljanjem arhitekture inventarja storitev, ki nastopi po implementaciji inventarja.

Kanonični izrazi (Canonical Expression)	• Kako je mogoče storitvene pogodbe dosledno razumeti in razlagati?
Kanonično verzioniranje (Canonical Versioning)	• Kako so lahko upravljamo različice storitvene pogodbe storitve znotraj istega inventarja storitev z minimalnim vplivom?
Centralizacija metapodatkov (Metadata Centralization)	• Kako se lahko metapodatki storitev centralno objavljajo in upravljajo?

30

2. Načrtovalski vzorci storitev

Sklop načrtovalskih vzorcev, ki so povezani z načrtovanjem storitev in arhitekture storitev

Temeljni vzorci
storitev

Vzorci
implementacije
storitev

Vzorci za varnost
storitev

Vzorci
načrtovanja
storitvenih
pogodb

Vzorci za
enkapsulacijo
podedovanih
sistemov in virov

Vzorci za
upravljanje in
nadzor storitev

31

Temeljni storitveni vzorci

Množica osnovnih vzorcev načrtovanja, ki pomagajo vzpostaviti temeljne značilnosti načrtovanja storitev.

Agnostična zmogljivost
(Agnostic Capability)

- *Kako lahko zagotovimo logiko storitve večnamensko, ki se učinkovito uporablja in sestavlja?*

Agnostični kontekst storitve
(Agnostic Context)

- *Kako lahko večnamensko storitveno logiko definiramo kot učinkovit poslovni vir?*

Ne-agnostični kontekst storitve
(Non-agnostic Context)

- *Kako se lahko enonamenska storitvena logika postavi kot učinkovit vir podjetja?*

Enkapsulacija storitev
(Service Encapsulation)

- *Kako je lahko logika rešitve na voljo kot vir podjetja?*

32

Vzorci implementacije storitev

Zbirka specializiranih načrtovalskih vzorcev, ki nudijo načrtovalske rešitve za različne izzive, povezane z zagotavljanjem arhitekture storitev.

Delna odložitev stanja (Partial State Deferral)	• Kako lahko preložimo določanje stanja, da zmanjšamo kompleksnost procesiranja in izboljšamo odzivnost storitve?
Delna validacija (Partial Validation)	• Kako lahko optimiziramo proces validacije, da pospešimo odziv in omogočimo postopno preverjanje podatkov?
Redundantna implementacija (Redundant Implementation)	• Kako lahko zagotovimo večjo zanesljivost in razpoložljivost storitve s podvajanjem funkcionalnosti?
Repliciranje podatkov storitve (Service Data Replication)	• Kako lahko izboljšamo dostopnost in zmogljivost storitev z repliciranjem podatkov?
Storitvena fasada (Service Facade)	• Kako lahko poenostavimo dostop do kompleksne logike z uporabo fasadne plasti, ki zmanjšuje odvisnosti?
Posrednik za uporabniški vmesnik (UI Mediator)	• Kako lahko zagotovimo prilagodljivo povezavo med uporabniškim vmesnikom in storitvami v ozadju?
Centralizacija referenčnih podatkov (Reference Data Centralization)	• Kako lahko zagotovimo konsistentnost in enotnost referenčnih podatkov v različnih storitvah?
Namestitev mikrororitev (Microservice Deployment)	• Kako lahko omogočimo skalabilnost in neodvisnost storitev z uporabo mikrororitvene arhitekture?
Nameščanje in uporaba čez zabojnike (Containerization)	• Kako lahko poenostavimo nameščanje in upravljanje storitev z uporabo zabojnikov, ki omogočajo izolacijo in prenosljivost?

33

Vzorci za varnost storitev

Vzorci se v glavnem ukvarjajo z oblikovanjem notranje logike storitve, ki nudi varnostne kontrole za preprečevanje pogostih varnostnih groženj.

Zaščita izjem (Exception Shielding)	• Kako lahko zaščitimo storitev pred nepričakovanimi izjemami, da zmanjšamo varnostna tveganja in zagotovimo stabilnost sistema?
Pregledovanje sporočil (Message Screening)	• Kako lahko validiramo in pregledamo sporočila, preden vstopijo v storitev, da zmanjšamo možnosti varnostnih groženj?
Varovanje obsega storitev (Service Perimeter Guard)	• Kako lahko omejimo dostop do storitev in zaščitimo obseg storitve pred neželenimi interakcijami?
Zaupanja vreden podsistem (Trusted Subsystem)	• Kako lahko zaupamo podsistemom, da omogočimo varne interakcije med storitvami in zunanji sistemi?

34

Vzorci storitvene pogodbe

Nabor načrtovalskih vzorcev, osredotočenih na načrtovanje storitvenih pogodb: vsebina + arhitekturni vidik.

Sočasne pogodbe (Concurrent Contracts)

- Kako lahko omogočimo hkratno uporabo različnih različic pogodb, da izboljšamo kompatibilnost storitev?

Centralizacija pogodb (Contract Centralization)

- Kako lahko centraliziramo pogodbe, da zmanjšamo redundantne definicije in poenostavimo vzdrževanje?

Denormalizacija pogodb (Contract Denormalization)

- Kako lahko prilagodimo strukturo pogodb, da povečamo učinkovitost in poenostavimo implementacijo storitev?

Odklopljena pogodba (Decoupled Contract)

- Kako lahko odklopimo pogodbe od specifičnih izvajanj, da povečamo fleksibilnost in združljivost storitev?

Abstrakcija overitve (Validation Abstraction)

- Kako lahko ustvarimo generično plast za overjanje, da omogočimo ponovno uporabo validacijskih pravil?

35

Vzorci za enkapsulacijo podedovanih sistemov in virov

Načrtovalski vzorci, ki se ukvarjajo s tem, kako ustrezno izpostaviti in zagotoviti interakcijo z obstoječimi podedovanimi sistemi in viri.

Datotečni prehod (File Gateway)

- Kako lahko omogočimo komunikacijo z zalednimi sistemi, ki temeljijo na izmenjavi datotek?

Ovojnica do zalednih/podedovanih sistemov (Legacy Wrapper)

- Kako lahko obdržimo funkcionalnost dedovanih sistemov, medtem ko omogočimo dostop do njih prek sodobnih storitev?

Večkanalna končna točka (Multi-Channel Endpoint)

- Kako lahko omogočimo dostop do storitev prek različnih komunikacijskih kanalov in naprav?

36

Vzorci za upravljanje in nadzor storitev

Za storitve, ki so že nameščene in v uporabi, ti vzorci obravnavajo pogoste izzive upravljanja, ki so tipično povezani z naknadno uvedbo sprememb.

Kompatibilne spremembe (Compatible Change)

- Kako lahko izvedemo spremembe v storitvi, ne da bi s tem prekinili obstoječe odvisnosti?

Razčlenjena zmogljivost (Decomposed Capability)

- Kako lahko razdelimo kompleksne zmogljivosti na manjše enote, da povečamo prilagodljivost in ponovno uporabo?

Porazdeljena zmogljivost (Distributed Capability)

- Kako lahko distribuiramo storitvene zmogljivosti, da izboljšamo skalabilnost in odzivnost?

Namestniška zmogljivost (Proxy Capability)

- Kako lahko uporabimo posredniško zmogljivost za zaščito dostopa do drugih storitev?

Dekompozicija storitev (Service Decomposition)

- Kako lahko razbijemo storitve na manjše komponente, da omogočimo bolj granularno ponovno uporabo in fleksibilnost?

Preoblikovanje storitev (Service Refactoring)

- Kako lahko optimiziramo obstoječe storitve brez prekinitev, da izboljšamo strukturo in zmogljivost?

Obveščanje o prekinitvah veljavnosti (Termination Notification)

- Kako lahko omogočimo obveščanje uporabnikov storitve o predčasni preinitvi storitve?

Identifikacija verzij (Version Identification)

- Kako lahko upravljamo različne verzije storitev, da zagotovimo združljivost in sledljivost?

37

3. Načrtovalski vzorci za kompozicije storitev

Sklop načrtovalskih vzorcev, ki so povezani z načrtovanjem kompozicij storitev ter interakcije s kompozicijami storitev

Vzorci
kompozicije
zmožnosti

Vzorci
sporočanja
storitev

Vzorci za
implementacijo
kompozicij

Vzorci za
varovanje
komunikacije
storitev

Vzorci
transformacij

38

Vzorci kompozicije zmožnosti

Temeljna vzorca, ki vzpostavljata osnovo za zmožnost storitve sestavljanja, ki se nanaša na oblikovanje in arhitekturo kompozicije.

Sestavljanje zmožljivosti (Capability Composition)

- Kako lahko združimo različne zmožljivosti v bolj kompleksne poslovne funkcionalnosti?

Ponovno sestavljanje zmožljivosti (Capability Recomposition)

- Kako lahko razporedimo obstoječe zmožljivosti, da izboljšamo prilagodljivost in zmožljivost storitve?

39

Vzorci sporočanja storitev

Nabor vzorcev se osredotoča na izmenjavo sporočil in procesiranje ter zagotavlja načrtovalske rešitve za širok nabor izzivov, povezanih s sporočanjem

Asinhrona vrsta (Asynchronous Queuing)

- Kako lahko omogočimo asinhrono obdelavo sporočil, da izboljšamo skalabilnost in obvladamo obremenitve?

Dogodkovno-usmerjeno sporočanje (Event-Driven Messaging)

- Kako lahko omogočimo obdelavo dogodkov, da izboljšamo odzivnost na spremembe in omogočimo obveščanje?

Vmesno usmerjanje (Intermediate Routing)

- Kako lahko optimiziramo usmerjanje sporočil med storitvami, da povečamo učinkovitost komunikacije?

Metapodatki sporočanja (Messaging Metadata)

- Kako lahko vključimo metapodatke v sporočila, da omogočimo natančnejšo obdelavo in upravljanje?

Zanesljivo sporočanje (Reliable Messaging)

- Kako lahko zagotovimo, da bodo sporočila uspešno dostavljena tudi v primeru napak ali prekinitev?

Storitveni agent (Service Agent)

- Kako lahko omogočimo posredovanje sporočil med storitvami prek posredniške entitete?

Povratni klic storitve (Service Callback)

- Kako lahko omogočimo storitvam, da se odzovejo na prejšnje zahteve s povratnimi klici?

Usmerjanje instance storitve (Service Instance Routing)

- Kako lahko usmerimo sporočila do specifičnih instanc storitev, da omogočimo učinkovito uporabo virov?

Storitveno sporočanje (Service Messaging)

- Kako lahko omogočimo komunikacijo med storitvami, ki temelji na protokolu sporočanja?

Sporočanje stanja (State Messaging)

- Kako lahko omogočimo posodabljanje stanja storitev, da izboljšamo koordinacijo in skladnost?

40

Vzorci za implementacijo kompozicij

Vzorci, ki se ukvarjajo predvsem z načrtovanjem arhitekture kompozicije in integriteto izvajanja kompozicij

Agnostični pod-kontrolnik (Agnostic Sub-Controller)

- Kako lahko omogočimo podkontrolnike, ki so prilagodljivi in neodvisni od specifičnih funkcij storitev?

Atomarna storitvena transakcija (Atomic Service Transaction)

- Kako lahko zagotovimo, da so transakcije storitev nedeljive in dokončne?

Kompenzacijska storitvena transakcija (Compensating Service Transaction)

- Kako lahko ustvarimo kompenzacijske transakcije za povrnitev stanja ob morebitni napaki?

Kompozicijska avtonomija (Composition Autonomy)

- Kako lahko zagotovimo neodvisnost storitvene kompozicije za izboljšanje robustnosti?

41

Vzorci za varovanje komunikacije storitev

Množica vzorcev, ki so osredotočeni predvsem na varnostne izzive, povezane z izvajanjem interakcij s storitvami in izmenjavo podatkov

Preverjanje pristnosti s posredovanjem (Brokered Authentication)

- Kako lahko omogočimo avtentikacijo prek posrednika, da zmanjšamo število neposrednih povezav?

Zaupnost podatkov (Data Confidentiality)

- Kako lahko zagotovimo zaupnost podatkov med prenosom, da zaščitimo občutljive informacije?

Potrjevanje izvora podatkov (Data Origin Authentication)

- Kako lahko preverimo izvor podatkov, da zagotovimo pristnost informacij?

Neposredno overjanje (Direct Authentication)

- Kako lahko omogočimo neposredno avtentikacijo uporabnikov brez posrednikov?

42

Vzorci transformacij

Načrtovalski vzorci, ki so povezani s transformacijo sporočil v času izvajanja čez vmesne nivoje procesiranja

Pretvorba podatkovnega formata

(Data Format Transformation)

- Kako lahko omogočimo pretvorbo podatkovnih formatov, da omogočimo združljivost med različnimi sistemi?

Pretvorba podatkovnega modela

(Data Model Transformation)

- Kako lahko prilagodimo podatkovni model, da olajšamo integracijo med sistemi?

Premostitev protokola

(Protocol Bridging)

- Kako lahko omogočimo komunikacijo med sistemi, ki uporabljajo različne protokole?

43

4. Načrtovalski vzorci na osnovi REST

Sklop načrtovalskih vzorcev, ki so nastali na osnovi arhitekturnega stila REST

Povezovanje entitet
(Entity Linking)

Lahka končna točka
(Lightweight Endpoint)

Ponovno uporabna pogodba
(Reusable Contract)

Pogajanje o vsebini
(Content Negotiation)

Preusmeritev končne točke
(Endpoint redirection)

Idempotentne zmožnosti
(Idempotent Capability)

44

5. Sestavljeni vzorci

Sestavljeni vzorec je grobo zrnat vzorec, sestavljen iz množice drobnozrnatih vzorcev.

Vodilo kanoničnih shem (Canonical Schema Bus)

Storitveno vodilo (Enterprise Service Bus)

Nivo zveznih končnih točk (Federated Endpoint Layer)

Uradna končna točka (Official Endpoint)

Orkestracija (Orchestration)

Storitveni posrednik (Service Broker)

Tri-nivojski inventar (Three-layer Inventory)

Enotna pogodba (Uniform Contract)

45

Arhitekturni stil REST

46

HTTP in REST

- HTTP – bogat aplikacijski protokol
 - *sinhroni "request/response-based" aplikacijski mrežni protokol za porazdeljene, sodelujoče, dokumentno-temelječe sisteme*
 - osnovni protokol spleta
- Spletne storitve na osnovi SOAP (in WS-*) sicer tudi uporabljajo HTTP kot prenosni protokol
 - **a ne izkoriščajo vseh zmožnosti HTTP**
- REST - ponovno "rojstvo" protokola http

47

REST

Representational State Transfer

ali

prenos stanj kot predstavitev

- Izraz REST (Representational State Transfer) je Roy Fielding definiral leta 2000 v svoji doktorski disertaciji.
 - bil je eden izmed soavtorjev protokola HTTP in je imel zelo dober vpogled v delovanje in razvoj spleta.
 - V disertaciji se je ukvarjal z vprašanjem, kako razširiti in uporabljati HTTP ter ostale spletne standarde za gradnjo porazdeljenih sistemov.
- Rezultat raziskav je **arhitekturni slog REST**
 - množica arhitekturnih principov, namenjenih zasnovi enostavnih, razširljivih in šibko sklopljenih porazdeljenih sistemov (s poudarkom na virih, reprezentacijah in uporabi standardnih HTTP metod).



48

REST - arhitekturni stil

- **REST je arhitekturni stil**
- **NI STANDARD!**
- Je **neodvisen od protokola komunikacije**
 - Res pa je največkrat omenjen v kontekstu arhitekturnih rešitev preko protokola HTTP

49

SOAP vs REST

- REST se je začel široko uporabljati veliko kasneje od nastanka ideje
- REST je danes **najpogosteje uporabljen arhitekturni slog za spletne API-je**,
 - zlasti pri javno dostopnih storitvah in storitvah, povezanih z družbenimi omrežji ter mobilnimi aplikacijami.
- SOAP ostaja močno prisoten predvsem na področju **poslovnih informacijskih sistemov in integracijskih platform**, kjer so pomembni:
 - stroga pogodba o vmesniku (WSDL),
 - standardizirani razširitveni mehanizmi (npr. WS-Security, WS-ReliableMessaging)
 - ter kompleksnejši poslovni scenariji.

50

Arhitekturni principi REST

01

Naslovljivi viri
(addressable
resources)

02

**Enoten
vmesnik** za vire
(uniform and
constrained
interface)

03

**Predstavitvena
usmerjenost**
(representation-
oriented)

04

**Komunikacija
brez stanja**
(communicate
statelessly)

05

HATEOAS
(Hypermedia As
The Engine Of
Application
State)

51

1. Naslovljivi viri

- Vir (resource): je ključna abstrakcija informacij in podatkov
- Dostop do vsakega vira aplikacije je omogočen preko **unikatnih identifikatorjev** (URI)
- URI → enoličen niz znakov, ki identificira vir.

52

Splošna sintaksa URI

scheme://host:port/path?queryString#fragment

Scheme – protokol (http, https)

Host – DNS ime ali IP naslov

Port – opcijsko

Path – izraz – posamezni deli ločeni z /

Query string – seznam parametrov v obliki **ime=vrednost**, ločilo &

#fragment – določen del v dokumentu

- Primer:

http://epodjetje.com/kupci?priimek=Novak&postnastevilka=2000

53

Zakaj je URI pomemben?

- **Enolična identifikacija virov**

- URI (oz. URL) je osnovni mehanizem za prepoznavanje virov v spletu – vsak vir ima svoj naslov, ki ga razumejo tako ljudje kot stroji.

- **Familiarnost oz. »domačnost«**

- Uporabniki so navajeni URL-jev:

- **Interoperabilnost**

- vseprisoten protokol
- na odjemalcu ne potrebujemo posebnih knjižnic, specifičnih za ponudnika storitve ("vendor specific" knjižnice)
- le HTTP knjižnica na odjemalcu
- ni potrebno naložiti nadomestnih objektov (stub), kreiranih iz IDL ali WSDL

54

URI načrtovalski napotki/smernice

- V spletu je velik pomen na dobro zgrajenih URI
- Dobro zgrajeni URIji uporabnikom omogočajo **intuitivno, predvidljivo in enostavno interakcijo s spletno stranjo**
- Dobra shema URI omogoča, da se posamezni naslovi ugotovijo

55

URI načrtovalski napotki/smernice

- Naslovi **URI** naj bodo **kratki**
- Raba samostalnikov namesto glagolov – **samostalniki v poti določajo vir**
- **Akcije (glagoli) v metodah HTTP**, ne v URI
- Identifikatorje in hierarhijo virov izražamo preko poti
- Ključne identifikatorje virov raje prenašamo kot pozicijske parametre v poti kot pa v poizvedbenem nizu, npr.:

~~GET /knjiga?isbn=24&year=1998~~

GET **knjiga/24/1998**

56

HTTP protokol in enoten vmesnik

- Nič več „*action*“ (in podobnih) parametrov v URI

~~GET /book?isbn=24&action=delete~~

- HTTP metode

- **GET** (read-only, idempotentna, varna)
- **PUT** (idempotentna, ni varna)
- **DELETE** (idempotentna, ni varna)
- **POST** (edina, ki ni idempotentna in ni varna)
- HEAD (kot GET, a dobimo le kodo in zaglavja)
- OPTIONS (za pridobitev informacij o komunikacijskih opcijah z virom in zmožnostih strežnika)
- TRACE
- CONNECT
- PATCH

57

URI načrtovalski napotki/smernice

- URI lahko “prilepimo” postfix-e, zato da določimo tip vsebine oz. predstavitev vira, ki ga lahko odjemalec pričakuje

http://osnovni.vir + .xml, .json, itd.

- To je sicer praksa, a REST in HTTP spodbujata uporabo zaglavja **Accept** (pogajanje o vsebini).
- Ne spreminjamo URI-jev
 - uporabimo raje preusmeritev, če jih je že res potrebno spremeniti

58

Zbirke (collections) in posamezni viri

- **Zbirke** virov izražamo z **množino**:
 - GET /knjige - pridobi seznam knjig
 - POST /knjige - ustvari novo knjigo
- **Posamezni vir**:
 - GET /knjige/123 - pridobi knjigo z ID = 123
- **Akcije** (glagoli) v **metodah HTTP** - primer (CRUD in HTTP):
 - CREATE → POST /knjige
 - READ → GET /knjige ali GET /knjige/123
 - UPDATE → PUT /knjige/123
 - DELETE → DELETE /knjige/123

59

Verzije API-ja

- V praksi se uporablja verzioniranje v URI:
 - jasna ločitev starih/novih API-jev
 - manj tveganja pri razbijanju obstoječih odjemalcev
- Najpogostejša praksa (verzija v poti):
 - /api/v1/books**
 - /api/v2/books/123**
- Alternativno: verzija v zaglavju (bolj napredno):
 - Accept: application/vnd.myapi.v1+json**

60

2. Enoten vmesnik (Uniform Interface)

- **Za vse interakcije med odjemalcem in strežnikom zadostuje končen, standardiziran nabor operacij nad viri.**
- Odjemalci ne potrebujejo posebnih, storitvam prilagojenih operacij - vedno uporabljajo isti nabor operacij nad različnimi viri.
- HTTP je zaradi omejenega nabora standardnih metod (GET, POST, PUT, DELETE, ...) primeren, pogosto idealen protokol za implementacijo RESTful storitev.

61

HTTP metode

Metoda	Varna	Idempotentna	Tipična raba
GET	✓	✓	Branje vira
HEAD	✓	✓	Metapodatki brez vsebine
OPTIONS	✓	✓	Zmožnosti strežnika/vira
PUT	✗	✓	Ustvarjanje / polna posodobitev
DELETE	✗	✓	Brisanje vira
POST	✗	✗	Kreiranje, akcije, podvire
PATCH	✗	Običajno ✗	Delna posodobitev

PATCH *lahko* implementiramo idempotentno, ni pa nujno po specifikaciji.

62

Idempotentna : varna metoda

- Idempotentna metoda:
 - Metodo lahko pošljemo poljubno krat in bo rezultat enak, kot če bi jo poslali samo 1x.
 - GET, HEAD, PUT, DELETE, OPTIONS
- Varna metoda:
 - Metoda ne spremeni stanja na strežniku
 - GET, HEAD, OPTIONS (in pogosto tudi TRACE, čeprav se v praksi skoraj ne uporablja).

63

Idempotentna zahteva

- Idempotentno zahtevo lahko pošljemo večkrat brez "stranskih učinkov"

GET /knjiga	<i>Metoda je idempotentna, če je implementacija na strežniku taka, da več klicev zapored pusti sistem v enakem stanju kot en klic.</i>
PUT /narocilo/x	
DELETE /narocilo/y	
- Varne zahteve so idempotentne zahteve, ki ne spremenijo stanja na strežniku
 - GET /knjiga**
- REST API z množico idempotentnih operacij omogoča razvoj robustnih rešitev
 - tudi če pride do vmesnega izpada storitve, lahko odjemalec večkrat pokliče metodo, dokler strežnik ni ponovno vzpostavljen

64

Enoten vmesnik skozi metode HTTP

Izmed vseh HTTP metod so najpomembnejše 4 metode, ki ustrezajo 4 CRUD operacijam:

CRUD	HTTP metoda	Opis
CREATE	POST	Kreiranje novega vira POST se lahko sicer uporabi tudi za druge metode, če recimo odjemalci zaradi kakršnegakoli razloga ne morejo prožiti PUT
READ	GET	Pridobitev trenutnega stanja vira
UPDATE	PUT	Inicializacija/posodobitev stanja vira na določenem URI - posodobitev celotnega vira , ne samo določenih atributov vira!
DELETE	DELETE	Brisanje vira, URI nato ni več veljaven
PARTIAL UPDATE	PATCH	Delna posodobitev vira.

65

3. Predstavitvena usmerjenost

- Viri sami po sebi niso podatki, temveč abstraktni koncepti domenskega modela (npr. knjiga, uporabnik, naročilo).
- Te koncepte je treba na zahtevo pretvoriti v obliko, ki jo odjemalec lahko prejme in obdela.
- **Takšna oblika se imenuje predstavitev vira (representation).**
- Različni odjemalci potrebujejo različne predstavitve, na primer:
 - spletni brskalniki: HTML, JSON,
 - mobilne ali strežniške aplikacije: JSON, XML,
 - drugi specializirani odjemalci: npr. CSV, binarni formati
 - itd.

66

En vir : več predstavitev

- Ločitev vira od predstavitve omogoča izgradnjo **šibko sklopljenih rešitev** in omogoča interakcijo z različnimi odjemalci
- **Ali lahko za vir preko istega URI pridobimo različne predstavitve vira?**

67

Več predstavitev istega vira

- **Vire z več predstavitvenimi formati lahko izpostavimo na 2 načina:**
 1. Uporaba **URI postfix-a**
 2. Uporabimo funkcionalnost protokola HTTP: **pogajanje o vsebini**

68

"Vsiljen" format

- URI kaže na točno določen predstavitveni format z uporabo končnice (razširitve naslova vira)

GET /vir.html

GET /vir.xml

GET /vir.json

Pazi!

- To je pogosta praksa, in ne standard!

69

"Pogajanje" glede formata

Odjemalec ob HTTP zahtevi doda nabor formatov, ki jih razume oz. lahko sprejme:

GET /vir

Accept: text/html, application/xml, application/json

Strežnik izbere najprimernejšega in odgovori:

200 OK

Content-type: application/json

70

MIME tip

- **Multipurpose Internet Mail Extensions (MIME) format** definira, kaj je v vsebini sporočila
 - da lahko vsebino tega sporočila na ustrezen način procesiramo
- Sintaksa: **type/subtype**
- Pogosti primeri:

text/plain	application/postscript
text/html	image/jpeg
application/xml	image/gif
application/json	audio/midi
text/css	application/octet-stream

71

4. Komunikacija brez stanja

- IDEJA: **vse zahteve, ki prispejo na strežnik, so samozadostne**
 - Zahteve vsebujejo vse informacije, ki so potrebne za strežnik, da lahko procesira zahtevo
 - Strežnik ne hrani sej odjemalcev in nikoli ne upošteva prejšnjih zahtev → **zahteve so med seboj neodvisne**
 - Strežnik skrbi zgolj za stanje vira
 - Če obstaja potreba po podatkih, specifičnih za sejo, jih mora hraniti in za njih skrbeti odjemalec in jih prenesti po potrebi z vsako zahtevo
- Torej
 - **STANJE APLIKACIJE** (stvar odjemalca)
 - **STANJE VIRA** (vzdržuje ga strežnik)

72

4. Komunikacija brez stanja

- Prednosti:

- lažje horizontalno skaliranje (ker strežnik ne hrani seje),
- boljša izraba predpomnjenja (cache),
- manj odvisnosti od posameznega strežnika (večja odpornost sistema),
- preprostejša arhitektura na strežniški strani.

- Kompromis:

- zahteve so lahko daljše (ker je potrebno poslati identifikator uporabnika / žetone),
- včasih potrebujemo dodatno plast (npr. JWT, tokeni, session storage v odjemalcu),
- del odgovornosti za stanje se prenese na odjemalca ali zunanje storitve (npr. identity provider)

73

5. HATEOAS

(Hypermedia As The Engine Of Application State)

- Je princip, ki je svojevrsten za REST aplikacijske arhitekture oz. je princip, ki v veliki meri loči REST od ostalih spletnih aplikacijskih arhitektur

- Osnovna ideja HATEOAS:

- **Interakcija med odjemalcem in storitvijo se v celoti upravlja skozi hipermedijske dokumente, ki jih dinamično zagotavlja strežnik**



Od tod koncept o tem, da hipermedija deluje kot stroj, ki skrbi za upravljanje stanja aplikacije

- **Podatkovni format vira zagotavlja vse informacije o tem, kako spremeniti/upravlja stanje aplikacije v določenem koraku**

74

HATEOAS

- Veliko "REST API-jev", ki jih danes uporabljamo (GitHub, Stripe, itd.), se ne drži HATEOAS v strogi meri, ampak še vedno govorimo o "REST API-jih".
- HATEOAS je bolj idealiziran princip iz izvirne REST definicije.

75

REST storitve in HATEOAS

- RESTful storitve, ki temeljijo na XML, lahko uporabljajo za implementacijo principa HATEOAS format **Atom Syndication**:

<link> + atributi:

- **rel** - kratko logično ime
- **href** - URL
- **type** - tip/format, ki se izmenjuje

76

Primer (namesto vseh izdelkov, le pet naenkrat)

- Storitve namesto seznama vseh izdelkov vrne seznam, ki vsebuje le 5 izdelkov.
 - V dokumentu XML najdemo informacijo, kje se nahaja množica naslednjih 5
- Na primer:

```
<products>
  <link rel="next" href="http://example.com/webstore/products?startIndex=5"/>
  <product id="123">
    <name>headphones</name>
    <price>€16.99</price>
  </product>
  ...
</products>
```

77

Primer (naslednjih 5 izdelkov)

- S klicem <http://example.com/webstore/products?startIndex=5> pridobimo naslednjo množico izdelkov, skupaj z informacijo na prejšnjo in naslednjo množico 5 izdelkov:

```
<products>
  <link rel="previous" href="http://example.com/webstore/products?startIndex=1"/>
  <link rel="next" href="http://example.com/webstore/products?startIndex=10"/>
  <product id="5323">
    <name>Coca Cola</name>
    <price>$16.99</price>
  </product>
  ...
</products>
```

78

HATEOAS: primer 2

GET /racun/12345 HTTP/1.1

HTTP/1.1 200 OK

```
<?xml version="1.0"?>
<racun>
  <st_racuna>12345</st_racuna>
  <stanje currency="eur">100,00</stanje>
  <link rel="polog" href="/racun/12345/polog" />
  <link rel="dvig" href="/racun/12345/dvig" />
  <link rel="prenos" href="/racun/12345/prenos" />
  <link rel="zapri" href="/racun/12345/zapri" />
</racun>
```

79

HATEOAS: primer 2 (po dvigu 125€)

GET /racun/12345 HTTP/1.1

HTTP/1.1 200 OK

```
<?xml version="1.0"?>
<racun>
  <st_racuna>12345</st_racuna>
  <stanje currency="eur">-25,00</stanje>
  <link rel="polog" href="/racun/12345/polog" />
</racun>
```

80

REST - povzetek

- REST definira ključne arhitekturne principe
 - **zaradi katerih je splet zelo skalabilen in robusten.**
- REST ponuja **preprost, interoperabilen in fleksibilen način razvoja spletnih storitev in API-jev.**
- REST predstavlja **drugačen način razmišljanja o razvoju:**
 - osredotoča se na vire,
 - uporablja standardne HTTP metode,
 - izkorišča obstoječo infrastrukturo spleta (URI, cache, HTTP protokol).

81

RESTful spletne storitve

82

RESTful spletne storitve

- Spletne storitve, ki sledijo arhitekturnim principom **REST**
- HTTP je naravna izbira za implementacijo RESTful storitev, ker:
 - ima **omejen** in **enoten nabor operacij** (metod)
 - je po specifikaciji **protokol brez stanja** (stateless)
 - z uporabo **URI** omogoča enolično naslavljanje virov
- **HTTP sam po sebi še ni REST**, je pa zelo dobro prilagojen REST principom

83

V praksi ločimo

Nizek REST (low REST)

- GET za branje/idempotentne zahteve, POST za "vse ostalo"
- URI-ji niso nujno "lepi" (pogosto vsebujejo glagole, action parametre, ...)
- Odgovori so lahko poljubni MIME tipi (npr. HTML, XML, JSON ...)

Visok REST (high REST)

- Uporaba lepih, semantičnih URI-jev (viri, ne akcije)
- Konsistentna uporaba metod: GET, POST, PUT, DELETE (in po potrebi PATCH)
- Odgovori v strukturirani obliki (JSON, XML, ...)
- Uporaba HATEOAS, kjer je to smiselno

*Delitev na low/high REST je le **priročen opis prakse**, ne uraden standard.*

84

REST - izzivi

- **high REST vs. low REST**

- koliko dosledno sledimo REST principom v praksi

- **Ali so res dovolj samo 4 metode?**

- HTTP/1.1 definira metode: GET, HEAD, POST, PUT, DELETE, OPTIONS, TRACE, CONNECT
- kasneje dodan še PATCH (RFC 5789)

- **Preslikava sinhronih REST operacij na zaledne sisteme**

- problem, če so zaledni sistemi asinhroni ali dogodkovno vodeni
- potrebujemo vmesne čakalne vrste, webhooke ...

85

REST - izzivi

- Izziv je **ustrezno identificirati in locirati vire** v različnih domenah

- Pomanjkanje strogega standarda za REST API-je

- standardizirani so **URI, HTTP, MIME, HTML**
- **opis API-jev** (semantike/sintakse) je pogosto neformalen

- Za opis REST API-jev se uporabljajo neformalni de facto standardi:

- **OpenAPI/Swagger**, RAML, API Blueprint, ...

- Dokumentacija REST API-ja mora biti berljiva za človeka in stroj

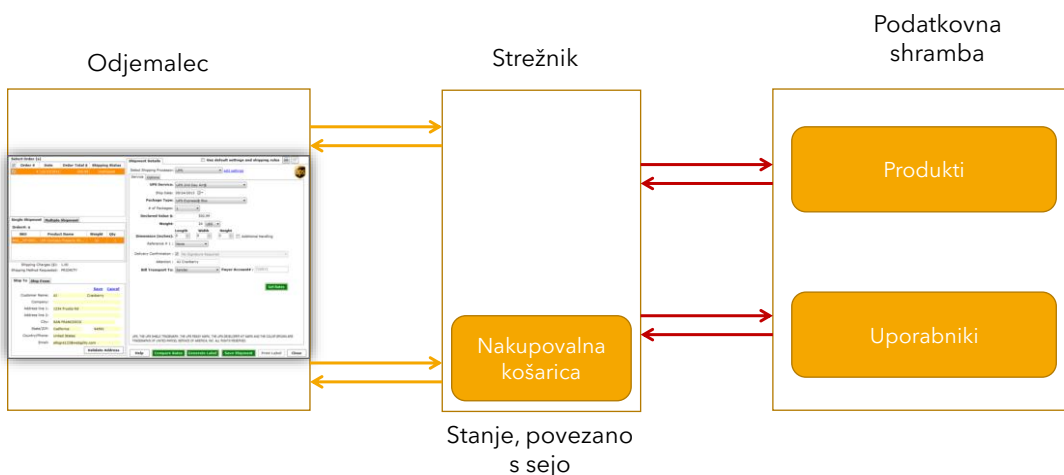
86

Upravljanje stanja na spletu?

- Ključno za doseganje principa "komunikacija brez stanja"
- Stanje aplikacije je na strani odjemalca
 - odjemalec si zapomni, "kje" v procesu je
- Stanje virov je na strežniku
 - strežnik skrbi za trajno shranjevanje in konsistentnost virov
- Vsaka zahteva vsebuje vse potrebne podatke (npr. identiteto uporabnika, identifikator košarice, ...)
- Ni "skrite" strežniške seje, odvisne od prejšnjih zahtev

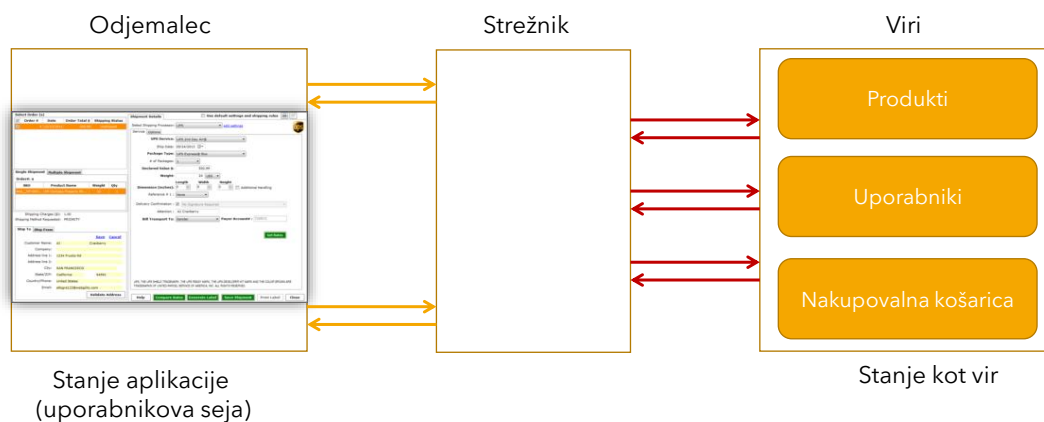
87

Stanje v klasični strežniški aplikaciji



88

Stanje kot vir



89

Nakupovanje brez stanja (primer)

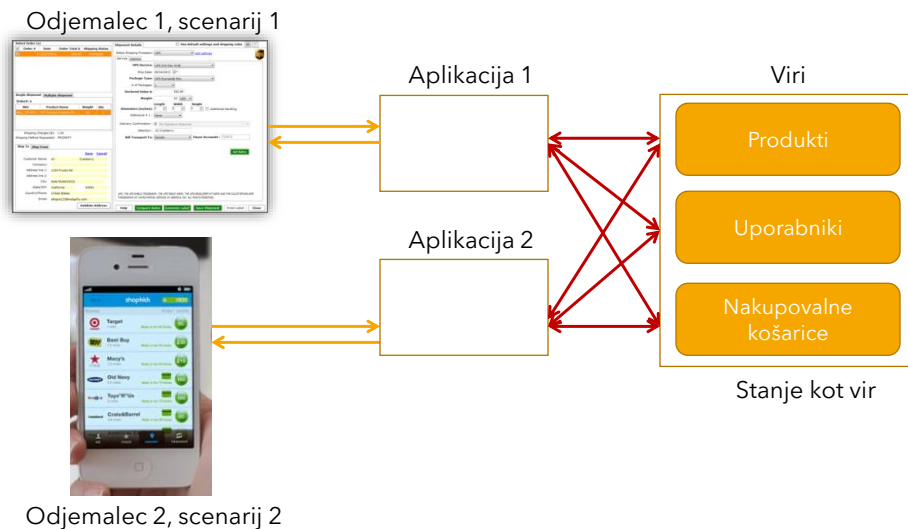
- Scenarije, ki vključujejo uporabo sejo, lahko preslikamo v zaporedje naslavljanja virov, na primer:

- Odjemalec: "Pokaži mi vse izdelke."
Strežnik: "Tu je seznam virov <http://enakup.com/izdeleki>."
- Odjemalec: "Dodaj 1 kos <http://enakup.com/izdelek/X> v mojo košarico, jaz sem Janez (pošlje uporabniško ime/geslo ali žeton)."
Strežnik: "Dodali smo izdelek X v <http://enakup.com/uporabniki/Janez/kosarica>."
- Odjemalec: "Dodaj 1 kos <http://enakup.com/izdelek/Y> v isto košarico, jaz sem Janez."
Strežnik: "Dodali smo izdelek Y v <http://enakup.com/uporabniki/Janez/kosarica>."
- Odjemalec: "Odstrani <http://enakup.com/izdelek/X> iz <http://enakup.com/uporabniki/Janez/kosarica>, jaz sem Janez."
Strežnik: "Odstranili smo X iz <http://enakup.com/uporabniki/Janez/kosarica>."
- Odjemalec: "Zaključim nakup, jaz sem Janez."
Strežnik: "Tu je znesek in račun za <http://enakup.com/uporabniki/Janez/kosarica>."

Ključna ideja: Ne gre le za to, da "sejo" modeliramo kot vir. Vsi relevantni podatki se trajno hranijo na strežniku kot viri (košarica, naročilo, račun ...). URI košarice ali naročila lahko uporabijo tudi druge storitve in odjemalci.

90

Uporaba virov v različnih odjemalcih



91

RPC (SOAP) vs
REST

**Storitve SOAP in REST ne
smemo mešati!**

SOAP / RPC

Usmerjenost v **operacije/storitve**
 Tipično za **integracijo poslovnih sistemov**
 Močno tipizirani vmesniki (WSDL, XSD)
 Kompleksnost v **API-jih/funkcijah**
 Pogosto skrito zaledje - manj vidni viri
(eden izmed principov je skrivanje)

REST

Usmerjenost v **vire**
 Pogosto za **sodelovanje in odprte API-je**
 Prožni, pogosto lahki formati (JSON, XML ...)
 Kompleksnost v **modelu virov in relacij**
 Odprti porazdeljeni viri, hiperlinki,
 HATEOAS

92

Metodologija načrtovanja in razvoja storitev REST

- 1. Identifikacija virov**, ki bodo izpostavljeni kot storitve (na primer letno poročilo poslovanja, katalog knjig, naročilo, odprte napake, itd.
- 2. Modeliranje povezav** (reference, prehodi stanj) med viri z uporabo povezav, ki jim lahko sledimo, da dobimo več podrobnosti (ali izvedemo prehode med stanji)
- 3. Definicija lepih URI** za naslavljanje virov
- 4. Razumevanje kaj pomeni GET, POST, PUT in DELETE za vsak vir** (in ali je dovoljeno uporabiti posamezno metodo ali ne)
- 5. Načrtovanje in dokumentiranje predstavitev virov**
- 6. Implementacija in nameščanje** na spletni strežnik
- 7. Testiranje** (spletni brskalnik, odjemalec, orodje kot je Fiddler,...)

	GET	PUT	POST	DELETE
/uporabnik	✓	✓	✓	✓
/naročilo	✓	✗	✗	✗
/knjiga	✓	✓	✓	✗
/kosarica	✓	✓	✓	✗
/racun	✓	✗	✓	✓
...				

93

Ustvarjanje virov

- Do REST storitve lahko **istočasno dostopa več odjemalcev**
 - več odjemalcev lahko na strežniku hkrati ustvarja nove vire
- Izziv:** kako zagotoviti enolične **identifikatorje virov**
- Možnosti:
 - ID določi **odjemalec** (npr. GUID, domensko pomemben ključ)
 - ID določi **strežnik** (avtoinkrement, UUID, ...)
- Za ustvarjanje virov uporabljamo:
 - PUT** (če odjemalec pozna ciljni URI)
 - POST** (če strežnik določi URI novega vira)

94

Ustvarjanje virov (PUT vs POST)

• PUT /vir/{id}

201 Created

Rešitev 1: odjemalec določi enoličen ID (na primer GUID)

• POST /vir

201 Created

Location /vir/{id}

Rešitev 2: Prepustimo strežniku, da v ozadju kreira enoličen id. Odjemalec potrebuje ID, če želi nadalje posodobiti ali iz brisati vir. HTTP zahteva, da v primeru, če POST kreira nov vir, odgovori s kodo 201 Created. V zaglavju v odgovoru na zahtevo pa se lahko zagotovi URI, tako da odjemalec ve, kje lahko v nadaljevanju komunicira z virom, ki ga je kreiral.

Opcijsko lahko pošlje strežnik še predstavitev vira

95

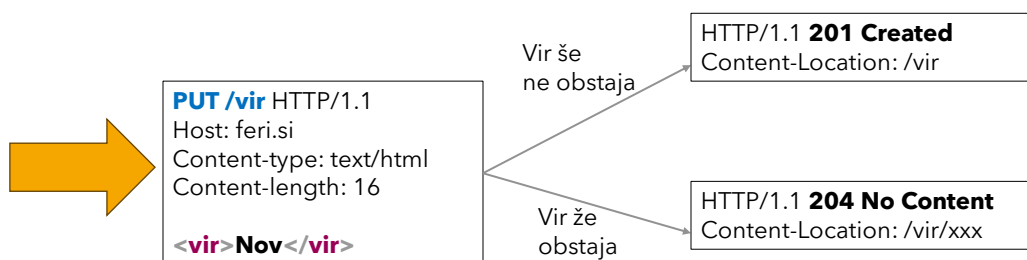
HTTP PUT vs POST

• PUT (idempotentna metoda)

- ustvari nov vir na točno določenem URI-ju, ali
- zamenja trenutno stanje ciljnega vira z novim stanjem iz telesa zahteve

• POST

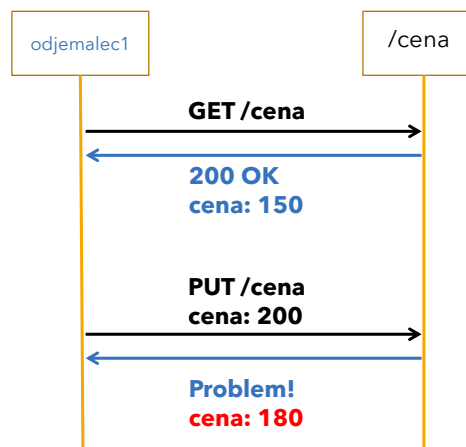
- ustvari nov vir pod danim URI-jem (npr. v zbirki)
- ni idempotentna – več klicev lahko ustvari več virov



96

Sočasen dostop do istega vira

- Kaj se zgodi, če dva odjemalca hkrati spreminjata isti vir?
- Scenarij:
 - Odjemalec 1 prebere vir (cena = 150) → 200 OK
 - Odjemalec 2 medtem spremeni vir (cena = 180) → 200 OK
 - Odjemalec 1 pošlje **posodobitev na podlagi starega stanja (150)**
- Če spremembo odjemalca 1 sprejmemo brez preverjanja, dobimo:
 - izgubo novejših podatkov (t. i. lost update)



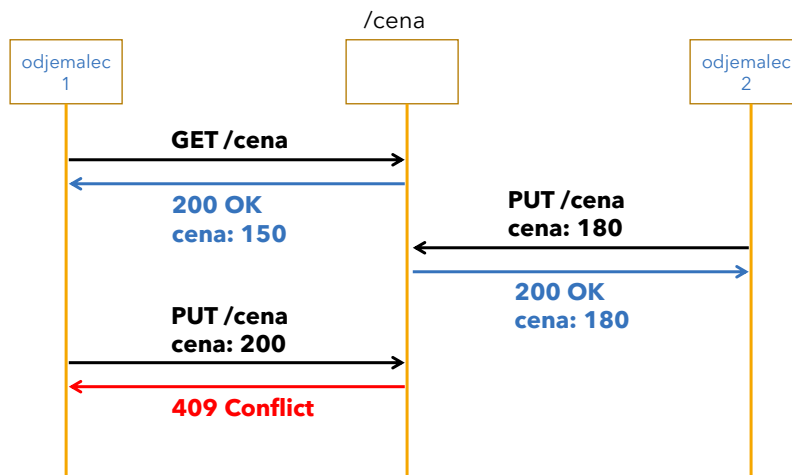
Potrebujemo **mehanizem za upravljanje sočasnih posodobitev**

97

Sočasen dostop do istega vira

- Eden od pristopov je, da strežnik **zavrne posodobitev**,
 - če ugotovi, da se je vir medtem že spremenil.
- V takem primeru strežnik vrne:

409 Conflict



98

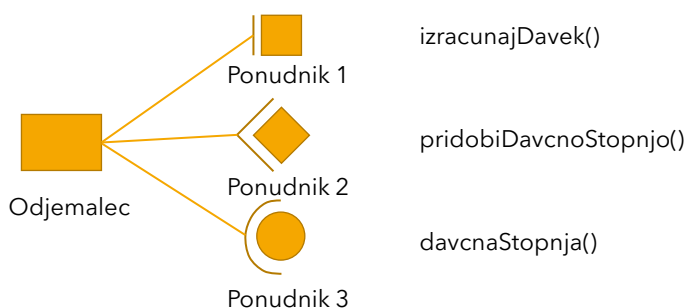
Načrtovalski vzorci in in anti-vzorci REST

Načrtovalski vzorci in pogoste napake pri zasnovi REST/HTTP programskih vmesnikov«



99

Vzorec: Enoten vmesnik (problem)



Praktični vzorec 'standardiziranega API-ja' med ponudniki, ne REST princip *uniform interface*.

• Problem:

- Tesna sklopljenost odjemalca in storitve.
- Odjemalec mora imeti nenehno posodobljene opise vmesnikov.
- Vsak ponudnik definira svoje metode in svoje parametre – odjemalec mora poznati vsakega posebej.
- Menjava ponudnika je draga, ker je odjemalca treba prilagoditi novemu vmesniku.

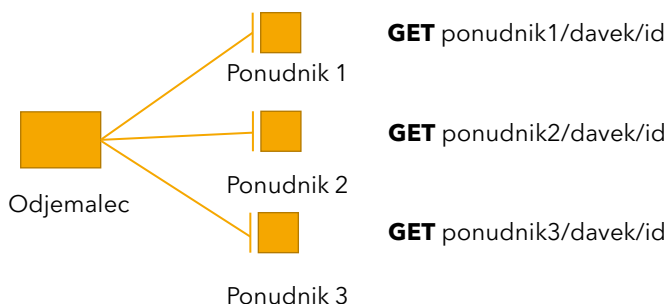
100

Vzorec: Enoten vmesnik (rešitev)

- **Rešitev:** Standardiziran enoten programski vmesnik čez končne točke različnih ponudnikov

- **Prednost:**

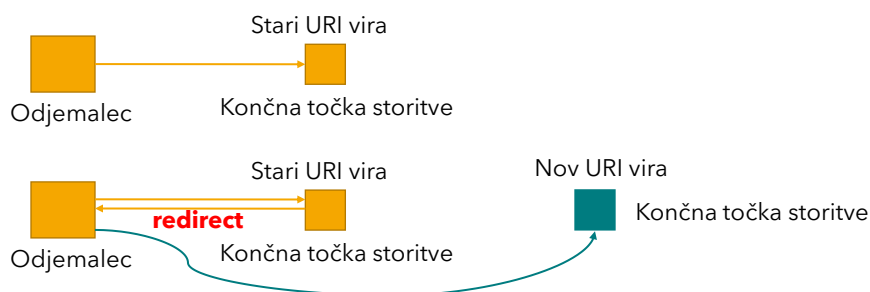
- Odjemalci uporabljajo enako strukturo URI-jev in enako podatkovno shemo ne glede na ponudnika;
- ob zamenjavi ponudnika običajno zamenjajo samo osnovni naslov (host)
- tipičen vzorec pri standardih panoge (bančništvo, e-računi ...), kjer se različni ponudniki zavežejo k istemu vmesniku.



vsi ponudniki izpostavijo **isti path in isti format odziva**, razlika je le v hostu.

101

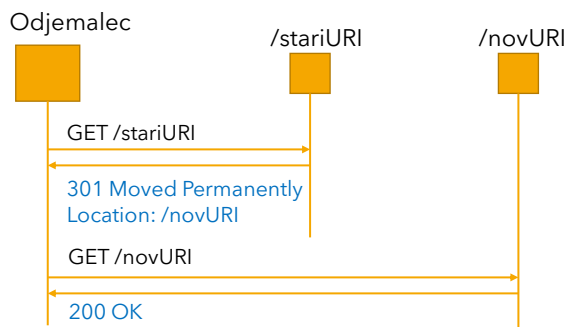
Vzorec: preusmeritev končne točke



- **Problem:** naslovi (URI-ji) storitev se lahko začasno ali trajno spremenijo zaradi različnih razlogov (poslovni, tehnični)
 - Ob vsaki spremembi URI-jev bi morali ročno posodobiti vse odjemalce, kar je drago in zamudno
 - Kako prilagoditi obstoječe odjemalce storitve, ko pride do spremembe na strežniški strani?
- **Rešitev:** HTTP obveščanje odjemalcev, ki dostopajo do storitev preko starih naslovov o novem URI

102

Vzorec: preusmeritev končne točke



- Protokol HTTP v osnovi omogoča vzorec preusmeritve končne točke z uporabo kombinacije statusnih kod 3xx in standardnih HTTP zaglavij

- 301 Moved Permanently
- 307 Temporary Redirect
- Location: /novURI

S tem lahko na strežni strani spreminjamo naslove storitev, odjemalci pa še naprej uporabljajo stare naslove in sledijo preusmeritvam.

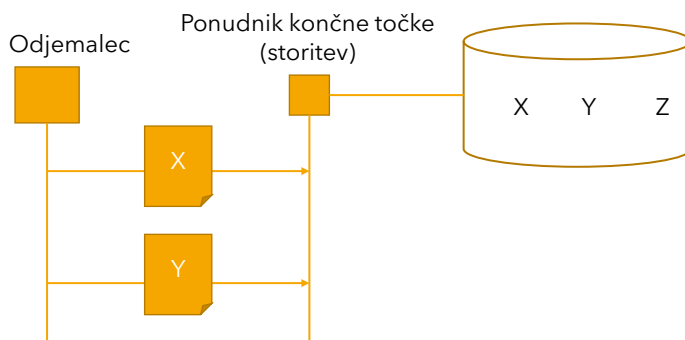
103

Vzorec: končna točka entitete

- **Storitve lahko v okviru funkcionalnosti delajo z različnimi entitetami!**

- **Problem:** Odjemalec mora poznati in uporabljati 2 ali več identifikatorjev

- **Globalni ID**, ki določa storitev in
- **Lokalni ID**, ki določajo entitete, povezane in upravljane preko storitve.



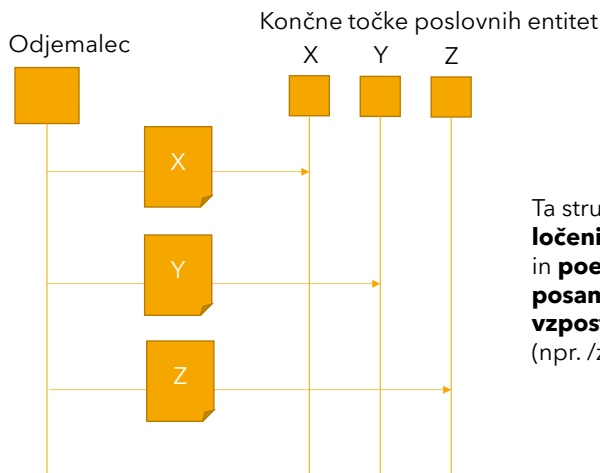
- Posledice:
 - oteženo predpomnjenje posameznih entitet,
 - težje uporabimo povezave (linke) na en sam vir

Odjemalec najprej naslovi splošno končno točko storitve (npr. /storitve/zaposleni)

- šele nato znotraj nje išče konkretno entiteto po lokalnem identifikatorju.
- URI konkretne entitete ni neposredno viden in ga težko delimo ali predpomnimo.

104

Vzorec: končna točka entitete

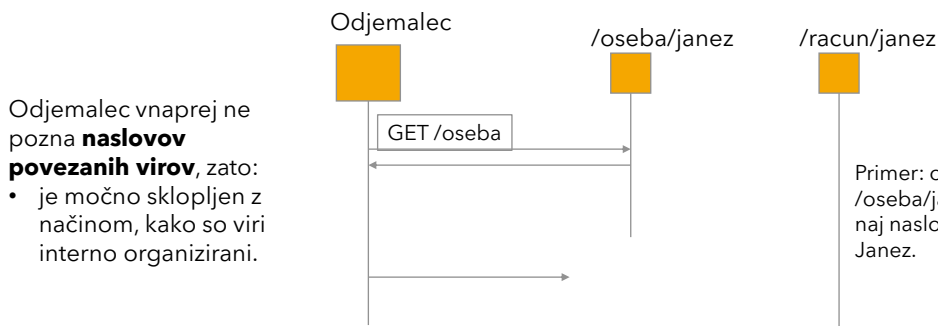


Ta struktura omogoča, da so **podatki ločeni po entitetah**, kar je **bolj pregledno** in **poenostavi predpomnjenje posameznih entitet** ter olajša **vzpostavljjanje povezav med entitetami** (npr. /zaposleni/123/narocila).

- **Rešitev: vsako poslovno entiteto izpostavimo na lastnem URI-ju**
 - Za posamezno entiteto (npr. zaposlenega) obstaja en sam, stabilen URI. Nad njim lahko uporabimo standardne metode GET, PUT, DELETE, ...

105

Vzorec: Povezovanje entitet



Odjemalec vnaprej ne pozna **naslovov povezanih virov**, zato:

- je močno sklopljen z načinom, kako so viri interno organizirani.

Primer: odjemalec pozna URI /oseba/janez, ne ve pa, kako naj naslovi račun osebe Janez.

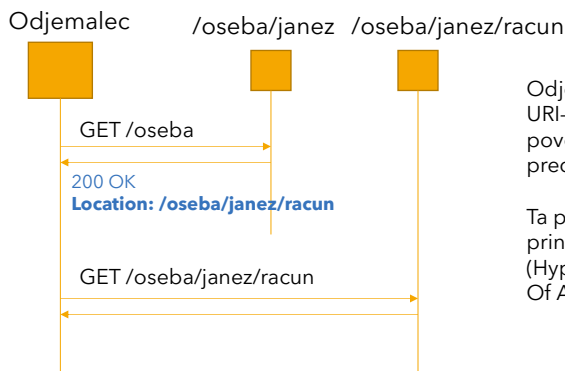
- **Problem:** Poslovne entitete so med seboj povezane (npr. oseba – račun – naročila).
- Če URI-ji teh povezav niso jasno definirani, odjemalec ne ve:
 - ali obstaja npr. /oseba/janez/racun,
 - ali mora klicati /racuni?uporabnik=janez,
 - ali je morda račun dostopen na /racun/123,
 - ...

106

Vzorec: Povezovanje entitet

- Odgovor za vir /oseba/janez lahko vsebuje:

- povezavo na račun, npr. /oseba/janez/racun,
- povezave na druga povezana stanja (naročila, košarica, profil ...).



Odjemalec tako ne ugiba URI-jev, ampak sledi povezavam, ki so del predstavitve vira.

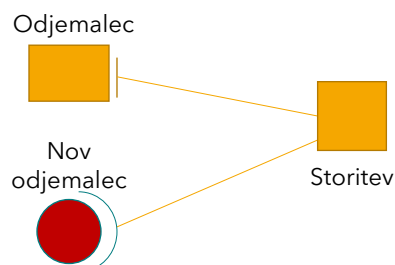
Ta pristop je v skladu s principom HATEOAS (Hypermedia As The Engine Of Application State).

- **Rešitev:** povezave do povezanih virov vključimo v predstavitev vira.
- Povezave lahko posredujemo:
 - v zaglavjih HTTP (npr. Location ali Link),
 - v telesu odgovora (npr. v JSON ali XML strukturi kot polje _links).

107

Vzorec: Pogajanje o predstavitvi

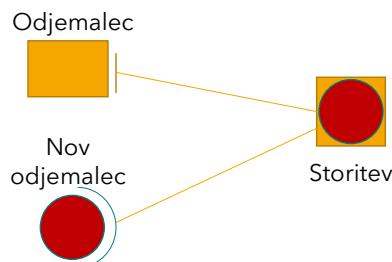
- **Problem:** Odjemalci imajo različne potrebe glede formata predstavitve:
 - brskalnik želi HTML,
 - spletna ali mobilna aplikacija želi JSON,
 - starejši odjemalci morda še vedno uporabljajo XML.
- Storitev želi podpirati več vrst odjemalcev,
 - ne da bi za vsak format uvajala ločene URI-je (npr. /vir.json, /vir.xml ...) in
 - ne da bi pri tem razbila obstoječe odjemalce.



108

Vzorec: Pogajanje o predstavitvi

- **Rešitev:** uporabimo pogajanje o predstavitvi (content negotiation) v HTTP.
- Odjemalec v zahtevi navede, katere medijske tipe (MIME tipe) podpira:
 - **Accept: application/json,**
 - **Accept: application/xml,**
 - **Accept: text/html,**
 - ...
- Storitev na podlagi zaglavja Accept:
 - izbere najprimernejši format,
 - ali vrne 406 Not Acceptable, če noben od zahtevanih formatov ni podprt.
- Pogodba storitve (specifikacija API-ja) jasno definira:
 - katere medijske tipe podpira,
 - morebitne razlike v vsebini med posameznimi formati.



Prednosti:

- manj podvajanja URI-jev,
- šibkejša sklopljenost med odjemalcem in strežnikom,
- lažje dodajanje novih formatov v prihodnosti.

109

Pogajanje o predstavitvi in HTTP

- HTTP neposredno podpira pogajanje o predstavitvi z zaglavji:
 - **Accept** - medijski tipi (npr. application/json, text/html),
 - **Accept-Language** - jezik predstavitve (npr. sl, en-US),
 - **Accept-Charset** - nabor znakov (npr. UTF-8),
 - **Accept-Encoding** - kodiranje vsebine (npr. gzip).

- Primer zahteve:

• GET /vir

Accept: text/plain, application/xml, application/json, text/html

Odjemalec navede tipe, ki jih zna razčleniti

• 200 OK

Content-type: application/xml

Strežnik izbere najprimernejši medijski tip in vrne odgovor, ali vrne 406 Not Acceptable, če nobena možnost ni podprta.

110

Več-parametrsko pogajanje

- Pogajanje o predstavitvi je lahko **večdimenzionalno**:
 - po **formatu** (JSON, XML, HTML),
 - po **jeziku** (Accept-Language: sl, en),
 - po **naboru znakov** (Accept-Charset: UTF-8),
 - po **kodiranju** (Accept-Encoding: gzip, deflate).

Zaglavje zahteve HTTP	Primeri vrednosti	Zaglavje odgovora HTTP
Accept:	application/xml, application/json	Content-type:
Accept-Language:	sl, en, es, de	Content-language
Accept-Charset:	iso-8859-2, unicode-utf-8	Charset
Accept-Encoding:	compress, gzip	Content-Encoding:

111

Vzorec: Porazdeljeno medpomnenje (Distributed Response Caching)

- Cilj: **enako predstavitev istega stanja vira ustvarimo čim manjkrat.**
- Predpomnjenje (caching) lahko izvajajo:
 - odjemalci (brskalniki, mobilne aplikacije),
 - posredni strežniki in povratni posredniki (proxy, CDN),
 - aplikacijski strežnik ali posebne predpomnilne plasti.
- Pri HTTP to omogočajo:
 - ETag** (entitetna oznaka),
 - Last-Modified**,
 - zaglavja **Cache-Control**, **Expires**, **Vary**.
- Sporočila HTTP ne vsebujejo samo podatkov, ampak tudi metapodatke, ki omogočajo:
 - zmanjšanje obremenitve strežnika,
 - hitrejše odzive,
 - učinkovito porazdeljeno predpomnjenje.

112

ETags

- **ETag (Entity Tag)** je oznaka določene predstavitve vira, ki jo določi strežnik.
- Uporablja se za:
 - preverjanje, ali se je vsebina vira spremenila,
 - učinkovito predpomnjenje in pogojne zahteve (conditional requests).
- V zaglavju odgovora strežnik doda npr.:

ETag: "88d979a0a78942b5bda05ace4214556a"

- Vrednost ETag lahko predstavlja:
 - izvleček (hash) vsebine,
 - interna oznaka različice,
 - kakšna druga enolična oznaka različice.
- Odjemalec ETag shrani in ga vključi pri nadaljnjih zahtevah

113

ETags – primer odgovora HTTP

HTTP/1.1 200 OK

Content-Type: application/json; charset=UTF-8

ETag: "88d979a0a78942b5bda05ace4214556a"

{ ... telo odgovora ... }

Strežnik:

- vrne običajen odgovor s telom (npr. JSON),
- v zaglavju doda entitetno oznako ETag.

Odjemalec:

- prebere vrednost ETag,
- jo shrani skupaj s predpomnjeno predstavitvijo vira.

114

ETags

- Ko odjemalec želi ponovno pridobiti vir, pošlje pogojno zahtevo::

GET /vir HTTP/1.1

Host: primer.si

Accept: application/json

If-None-Match: "88d979a0a78942b5bda05ace4214556a"

Z zaglavjem If-None-Match odjemalec strežniku sporoči:

- "Pošlji mi predstavitev samo, če se je spremenila glede na ETag, ki ga že imam."

Strežnik na osnovi tega določi:

- ali se je vir vmes spremenil ali ne,
- in izbere ustrezno statusno kodo ter sestavi odgovor.

115

ETags (odgovor strežnika)

- Če na strežniku ni prišlo do spremembe predstavitve vira:

HTTP/1.1 304 Not Modified

- brez telesa odgovora,
- odjemalec uporabi svojo predpomnjeno kopijo.

- Če se je predstavitev vira spremenila, strežnik vrne:

HTTP/1.1 200 OK

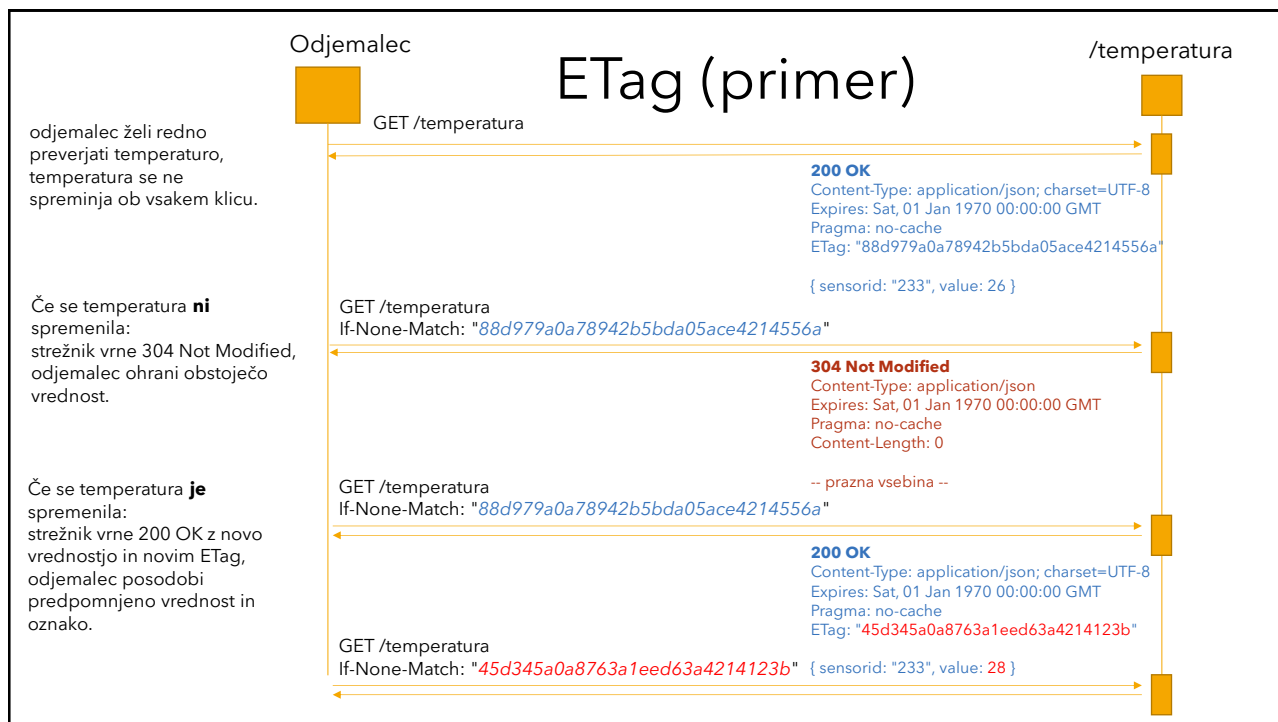
Content-Type: application/json

ETag: "nova-oznaka"

{ ... nova predstavitev vira ... }

- Odjemalec:
 - zamenja svojo lokalno kopijo,
 - posodobi shranjeni ETag.

116



117

Last-Modified

- Pri tem pristopu strežnik v zaglavju odgovora vedno zagotovi čas zadnje spremembe vira:

HTTP/1.1 200 OK

Content-Type: application/json; charset=UTF-8

Last-Modified: Mon, 25 Nov 2019 20:29:14 GMT

{ ... telo odgovora ... }

- Last-Modified je metapodatek, ki pove:
 - kdaj je bil vir nazadnje spremenjen (po mnenju strežnika),
 - omogoča pogoje zahteve na osnovi časa.

118

Last-Modified (naslednje zahteve)

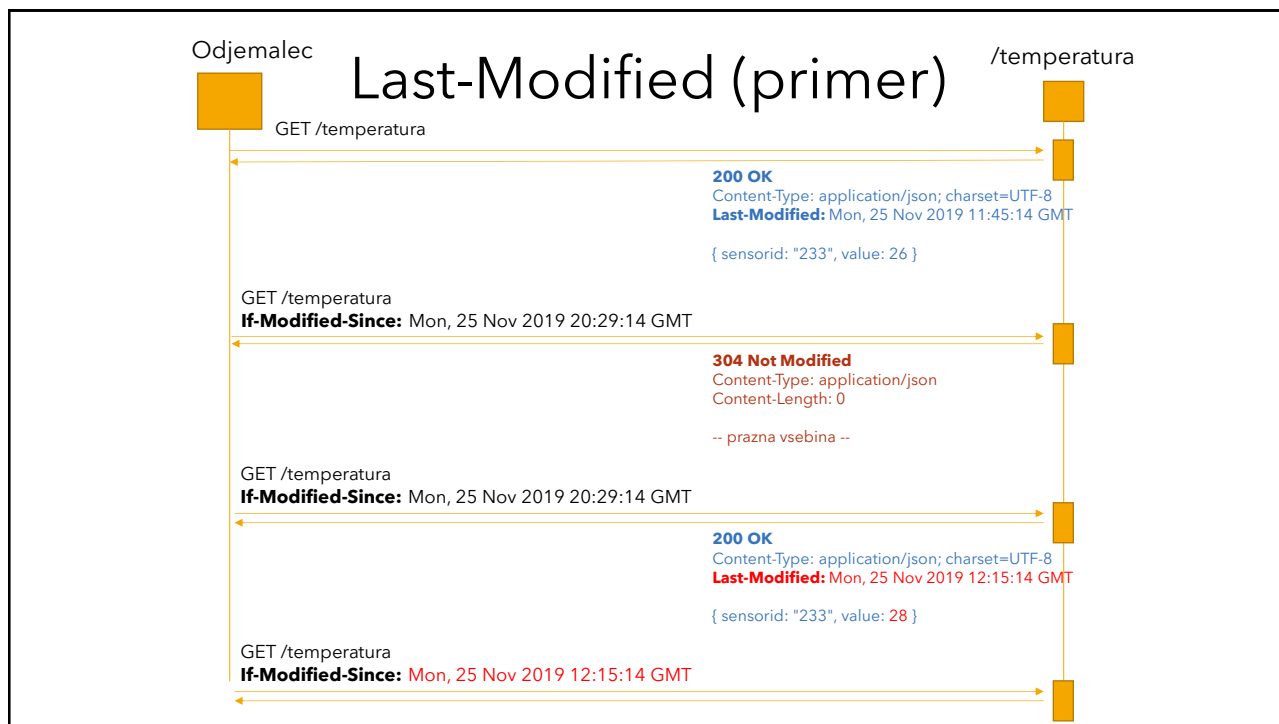
- Odjemalec pri naslednjih klicih uporabi zaglavje **If-Modified-Since:**
GET /vir HTTP/1.1
Host: primer.si
If-Modified-Since: Mon, 25 Nov 2019 20:29:14 GMT
- Strežnik preveri ali je vir spremenil stanje po tem času.
- Če vir **ni bil spremenjen**, strežnik vrne:
HTTP/1.1 304 Not Modified
- Če vir je bil spremenjen strežnik vrne nov odgovor **200 OK** z novo predstavitvijo in posodobljenim Last-Modified.

119

Last-Modified - posebnosti

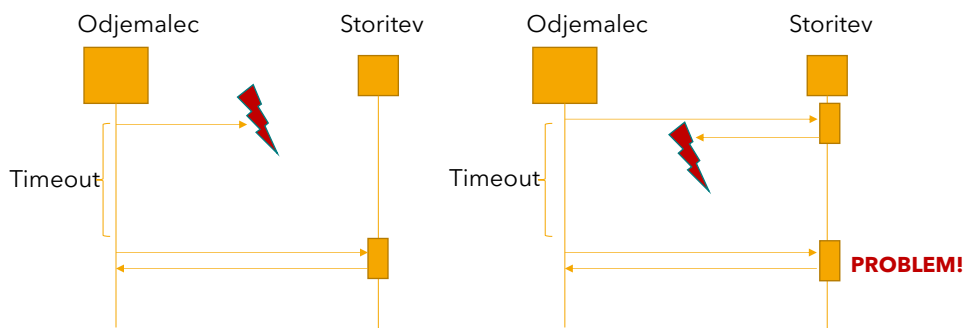
- Last-Modified ima tipične omejitve:
 - ločljivost je pogosto ena sekunda,
 - več sprememb v istem časovnem intervalu se lahko "skrije".
- V praksi:
 - je Last-Modified preprost in pogosto zadosten,
 - za natančnejši nadzor sprememb je bolj primeren ETag.
- Mehanizem je podoben ETag-u:
 - le da temelji na času zadnje spremembe, ne na poljubni oznaki.

120



121

Vzorec: idempotentne zmožnosti storitve



• **Problem:** V porazdeljenih sistemih lahko pride do:

- izgube sporočila ali odgovora,
- časovnih prekoračitev (timeout),
- ponovnega pošiljanja iste zahteve.

Če operacije niso idempotentne, ponovitev iste zahteve lahko povzroči

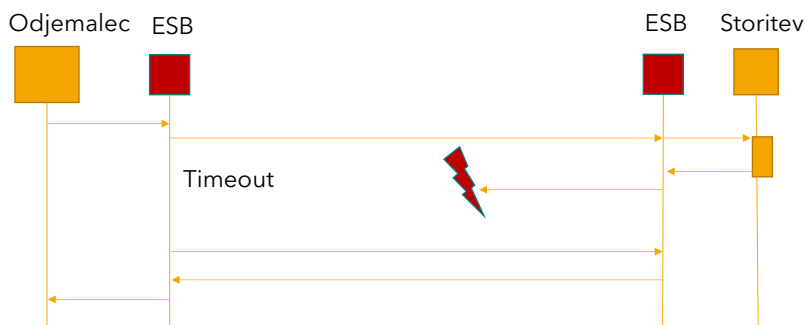
- podvojena naročila,
- dvojna knjiženja,
- druge neželene stranske učinke.

Odjemalec včasih ne more zanesljivo vedeti, ali je strežnik zahtevo že obdelal ali ne.

Potrebujemo način, kako omogočiti varno ponavljanje (retry), ne da bi ogrozili pravilnost poslovne logike.

122

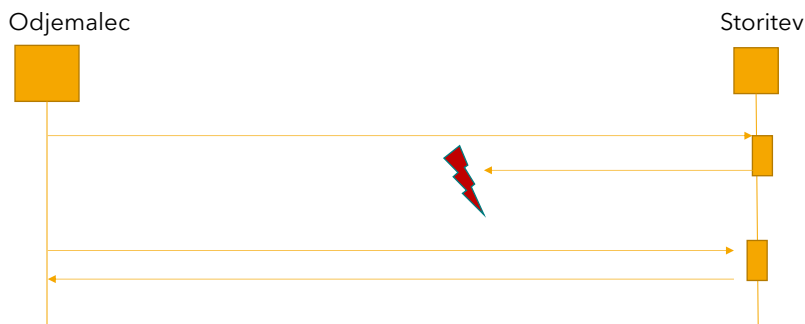
Rešitev: ESB?



- Ena izmed možnih rešitev je uporaba storitvenega vodila (Enterprise Service Bus - ESB):
 - zanesljivo posredovanje sporočil,
 - trajno shranjevanje sporočil,
 - vrste (queue), ponovni poskusi, usmerjanje.
- Omejitve:
 - dodatna kompleksnost in odvisnost od infrastrukture,
 - ESB sam po sebi ne zagotovi idempotentnosti – če operacija ni idempotentna, lahko ponovni poskusi še vedno povzročijo podvojene učinke.

123

Vzorec: idempotentne zmožnosti storitve



- **Enostavnejša rešitev:** Kjer je le mogoče, zasnujemo **operacije kot idempotentne**:
 - GET je po specifikaciji varen in idempotenten,
 - PUT in DELETE naj bodo implementirani idempotentno,
- Pravilo:
 - ponavljanje iste zahteve ne spremeni končnega stanja vira.

124

REST antivzorci – Tuneliranje HTTP GET

- Uporaba metode GET za operacije, ki spreminjajo stanje na strežnik. Primeri:

GET /api?method=dodajStranko&ime=Janez&priimek=Novak

GET /api?method=brisiStranko&id=4102

GET /api?method=vрниStranko&id=4102

GET /api?method=najdiStranke&lastName=Nova*

• Težave:

- kršitev semantike GET (ta naj ne bi imel stranskih učinkov),
- podatki v URI-ju se pojavljajo v dnevnikih, zgodovini, zaznamkih,
- omejena dolžina URI-jev v praksi (strežniki in posredniki).

• Posledica:

- gre za RPC preko HTTP, kjer se prava operacija skriva v parametru method,
- ni skladno z arhitekturnim slogom REST.

125

REST antivzorci – Tuneliranje HTTP POST

- Uporaba metode POST kot splošnega "tunela"
 - za poljuben lastni protokol (npr. SOAP), brez izkoriščanja semantike HTTP.
- Primer (SOAP ovojnica znotraj POST):

POST /storitev/koncnatocka

```
<soap:Envelope
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xmlns:xsd="http://www.w3.org/2001/XMLSchema"
xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/">
  <soap:Body>
    <najdiStranke xmlns="http://zajfa.si/">
      <priimek>Nova*</priimek>
    </najdiStranke>
  </soap:Body>
</soap:Envelope>
```

Težave:

- HTTP je uporabljen le kot prenosni protokol,
- semantika HTTP metod (GET, PUT, DELETE ...) ostane neizkoriščena,
- oteženo predpomnjenje in uporaba posrednikov,
- vsak odjemalec mora razumeti poseben protokol v telesu sporočila.
- To je tipičen primer oddaljenega klica procedur (RPC) nad HTTP, ne pa REST.

126

REST antivzorci - Tuneliranje napak skozi HTTP Response Body

- Storitve vedno vrača 200 OK, napake pa sporoča le v telesu odgovora.
Primer:

GET /artikel/12343

HTTP/1.1 200 OK

Content-Type: application/xml

```
<response code="error">
  <message>Artikel 12345 ne obstaja.</message>
</response>
```

↑
Odjemalec mora razumeti vsebino napake! Tesna sklopljenost s specifično metodo oz. implementacijo oblikovanja napak

Težave:

- odjemalec mora za napake parsati telo in ignorirati statusno kodo,
- infrastrukturne komponente (proxy, knjižnice HTTP) ne morejo
- izkoristiti standardnih statusnih kod,
- oteženo beleženje, nadzor in obravnava napak.
- Navzven deluje, kot da je bilo vse uspešno (200 OK), čeprav je prišlo do napake.

127

REST antivzorci - Tuneliranje napak skozi HTTP Response Body

Boljša rešitev: napake izražamo s pravnimi HTTP statusnimi kodami. Če npr. artikel ne obstaja, vrnemo:

GET /artikel/12343



HTTP/1.1 404 Not Found

Content-Type: application/json

```
{ "code": "ARTICLE_NOT_FOUND",
  "message": "Artikel 12345 ne obstaja." }
```

Prednosti:

- odjemalec lahko najprej pogleda statusno kodo,
- telo odgovora vsebuje podrobnejšo razlago napake,
- posredniki in orodja za beleženje lahko pravilno ravnajo z različnimi tipi napak.
- Standardne statusne kode (4xx, 5xx) uporabljamo skladno s specifikacijo HTTP.

128

Standardne HTTP kode

100 Continue
 200 OK
 201 Created
 202 Accepted
 203 Non-Authoritative
 204 No Content
 205 Reset Content
 206 Partial Content
 300 Multiple Choices
 301 Moved Permanently
 302 Found
 303 See Other
 304 Not Modified
 305 Use Proxy
 307 Temporary Redirect

Napake iz strani odjemalca:

400 Bad Request
 401 Unauthorized
 402 Payment Required
 403 Forbidden
 404 Not Found
 405 Method Not Allowed
 406 Not Acceptable
 407 Proxy Authentication Required
 408 Request Timeout
 409 Conflict
 410 Gone
 411 Length Required
 412 Precondition Failed
 413 Request Entity Too Large
 414 Request-URI Too Long
 415 Unsupported Media Type
 416 Requested Range Not Satisfiable
 417 Expectation Failed

Napake na strani strežnika:

500 Internal Server Error
 501 Not Implemented
 502 Bad Gateway
 503 Service Unavailable
 504 Gateway Timeout
 505 HTTP Version Not Supported

129

Microsoft REST API Guidelines Are Not RESTful

- V praksi se pojavljajo različne smernice in priporočila za **spletne API**-je, npr. navodila Microsofta.
- Velik del teh priporočil je namenjen:
 - doslednemu poimenovanju URI-jev,
 - uporabi standardnih HTTP metod,
 - enotni strukturi odzivov,
 - podpori več programskim jezikom in orodjem.
- Čeprav gre za koristna priporočila za **HTTP API**-je,
 - niso nujno skladna z vsemi arhitekturnimi omejitvami
 - sloga REST (npr. pogosto ne upoštevajo HATEOAS).

https://www.infoq.com/news/2016/07/microsoft-rest-api?utm_source=infoq&utm_medium=related_content_link&utm_campaign=relatedContent_articles_clk

130

Microsoft REST API Guidelines Are Not RESTful

- URLs should be readable and constructible by humans to facilitate the adoption of such APIs without the need of a client library.
- The following HTTP verbs are supported: GET, PUT, DELETE, POST, HEAD, PATCH, OPTIONS. Not all resources should support all verbs.
- GET, PUT, DELETE and HEAD should be idempotent.
- Custom headers must not be required.
- Clients should not be required to transmit personally identifiable information parameters in the URL because they might be exposed through log files. The parameters should be passed through headers.
- Data should be provided in popular formats.
- The default data encoding is JSON.
- "Primitive values MUST be serialized to JSON following the rules of RFC4627."
- Developers using the APIs should be able to use the language and platform of preference.
- Services should be accessible even from simple HTTP tools such as curl.
- APIs should support explicit versioning.
- Services should be stable without introducing breaking changes if possible, but they should be allowed to do so if necessary, incrementing the version number to signal that.
- Versioning should be done using a Major.Minor scheme.
- Services may implement push notifications via web hooks (HTTP Callbacks).

https://www.infoq.com/news/2016/07/microsoft-rest-api?utm_source=infoq&utm_medium=related_content_link&utm_campaign=relatedContent_articles_clk

131

Microsoft REST API Guidelines Are Not RESTful

Povzetek priporočil Microsoft (povzetek):

- URI-ji naj bodo berljivi in dosledni,
- uporabljamo standardne metode HTTP,
- napake izražamo s statusnimi kodami in strukturiranimi odgovori,
- API naj bo neodvisen od tehnologije odjemalca,
- API naj podpira asinhrono scenarije (npr. z zgodovino stanja, obvestili, web hooki).
- To so dobra praktična priporočila za zasnovo spletnih (Web) API-jev.
- Vendar ta navodila ne pokrivajo vseh posebnosti arhitekturnega sloga REST (npr. hipermedija kot gonilo stanja aplikacije).

https://www.infoq.com/news/2016/07/microsoft-rest-api?utm_source=infoq&utm_medium=related_content_link&utm_campaign=relatedContent_articles_clk

132

Odziv Roy Fieldinga na navodila Microsoft

- I think it is great that companies like Microsoft have decided to publish some of their internal documentation and development guidelines on public forums, especially within open collaboration spaces like GitHub. This openness to public conversations will significantly improve how we all work within this industry.
- What I dislike about these guidelines is that **they are a compilation about how one might develop reasonable HTTP APIs within the Microsoft ecosystem, but is labelled as being REST and RESTful APIs. REST != HTTP**, as is obvious from a cursory read of any of the relevant materials about REST. **The guidelines are clearly not based on REST** and don't even manage to reference my work (aside from the parts of RFC2616 that have been obsoleted by RFC7231). This is a common mistake by folks who were formerly deep in the Web Services world: **to describe REST as some sort of HTTP version of a SOAP/RPC interface**.
- No matter how common that mistake is in practice, it cannot pretend to be REST. Most of the properties of the REST architectural style come from adhering to all of its constraints, not just those that are conveniently similar to failed tools of the past. I have no problem with folks who only want to build RPC interfaces using HTTP, provided they call them HTTP APIs. They are not REST. They are not of the Web. That doesn't mean they can't be honestly described and implemented as HTTP services. It is worthwhile to understand them as such, and to discuss their implementation guidelines without feeling a need for mistaken buzzword compliance.

https://www.infoq.com/news/2016/07/microsoft-rest-api?utm_source=infoq&utm_medium=related_content_link&utm_campaign=relatedContent_articles_clk

133

Odziv Roy Fieldinga na navodila Microsoft

- Roy Fielding opozarja, da:
 - so smernice, kot so Microsoftove, koristne za HTTP API-je, vendar ne opisujejo nujno RESTful sistemov,
 - je REST arhitekturni slog z natančno definiranimi omejitvami, ne zgolj "API prek HTTP-ja".
- Njegovo sporočilo v grobem:
 - poimenovanje API-ja kot RESTful ima smisel le, če ta zares upošteva arhitekturne principe REST,
 - v nasprotnem primeru je bolj pošteno govoriti o HTTP API-ju ali spletnem API-ju brez oznake REST.

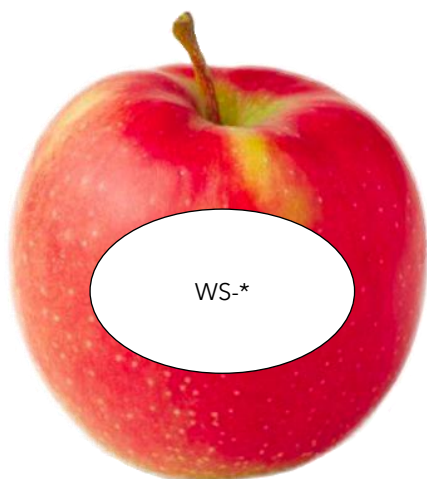
https://www.infoq.com/news/2016/07/microsoft-rest-api?utm_source=infoq&utm_medium=related_content_link&utm_campaign=relatedContent_articles_clk

134

(SOAP) WS-* vs REST

135

Ali lahko primerjamo
WS-* in REST?



136

Ali lahko primerjamo WS-* in REST?

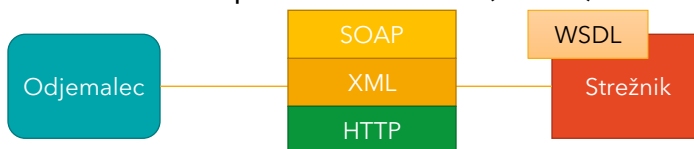


137

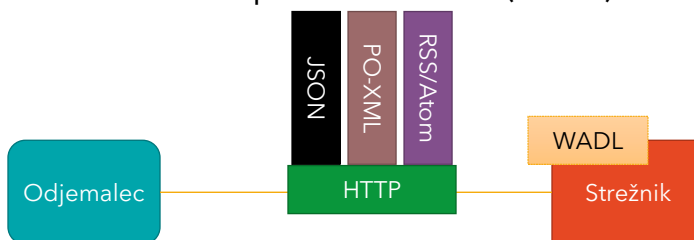
Spletne strani (splet) (1992)



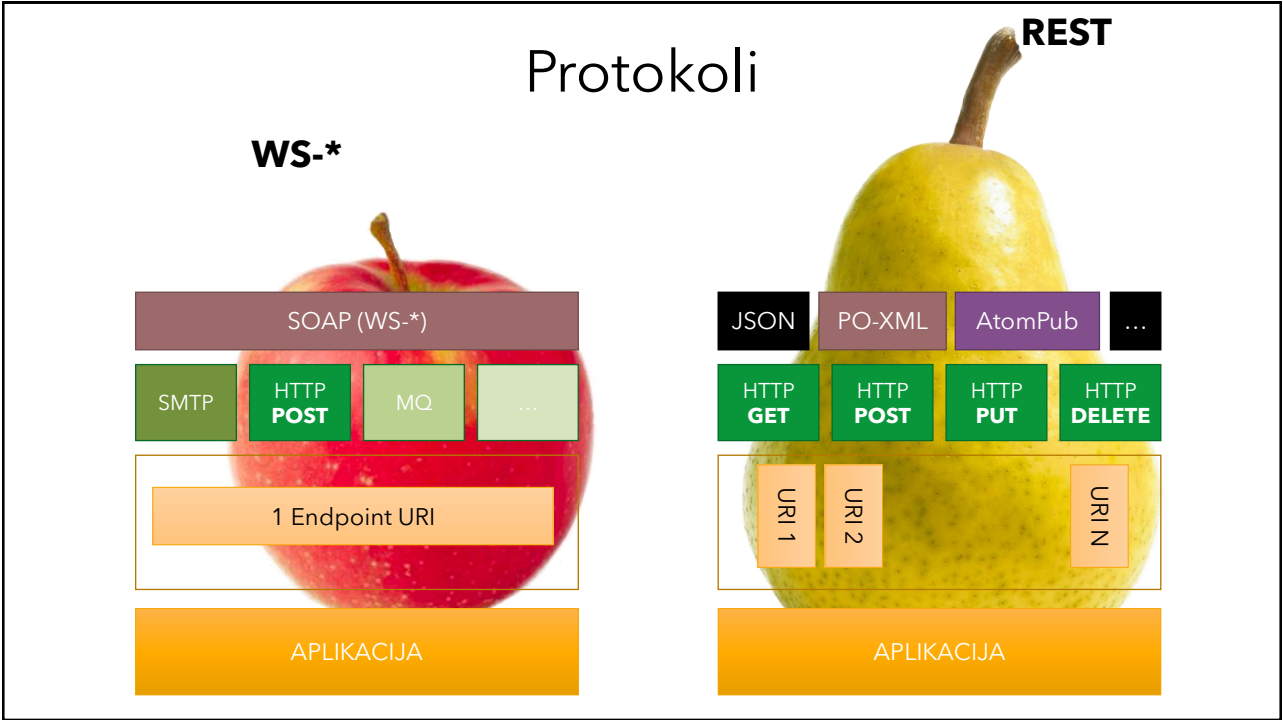
WS-* Spletne storitve (2000)



RESTful spletne storitve (2007)



138

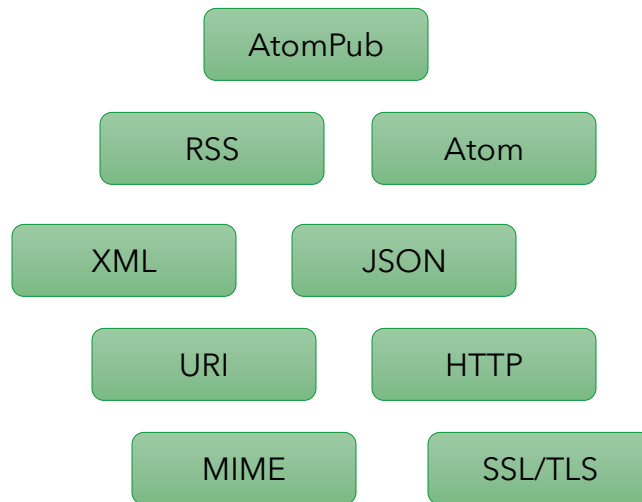


139

Sklad standardov WS-*		
Prenos	HTTP, IIOP, SMTP, JMS	
Sporočanje	XML, SOAP	WS-Addressing
Opis	XML Schema, WSDL	WS-Policy, SSDL
Odkrivanje	UDDI	WS-Metadata-Exchange
Koreografija	WSCL WSCI	WS-Coordination
Poslovni procesi	WS-BPEL BPML	WSCDL
Viri brez stanja	WS-Resource Framework	
Transakcije	WS-CAF	WS-Atomic Transactions WS-Business Activities
Zanesljivo sporočanje	WS-Reliability	WS-ReliableMessaging
Varnost	WS-Security SAML, XACML	WS-Trust, WS-Privacy, WS-SecureConversation
Obveščanje o dogodkih	WS-Notification	WS-Eventing
Upravljanje	WSDM	WS-Management
Dostop do podatkov	OGSA-DAI	SDO

140

Sklad standardov za RESTful spletne storitve



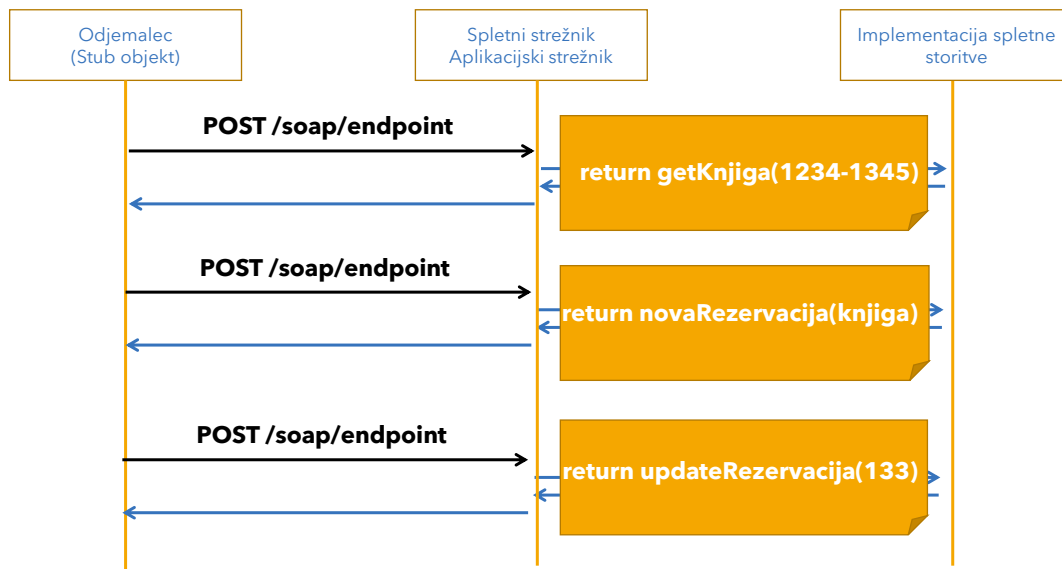
141

WS-* vs REST storitve

- Spletne storitve (WS-*) – **action based, operation based**
- REST – **resource based** (Representational State Transfer)

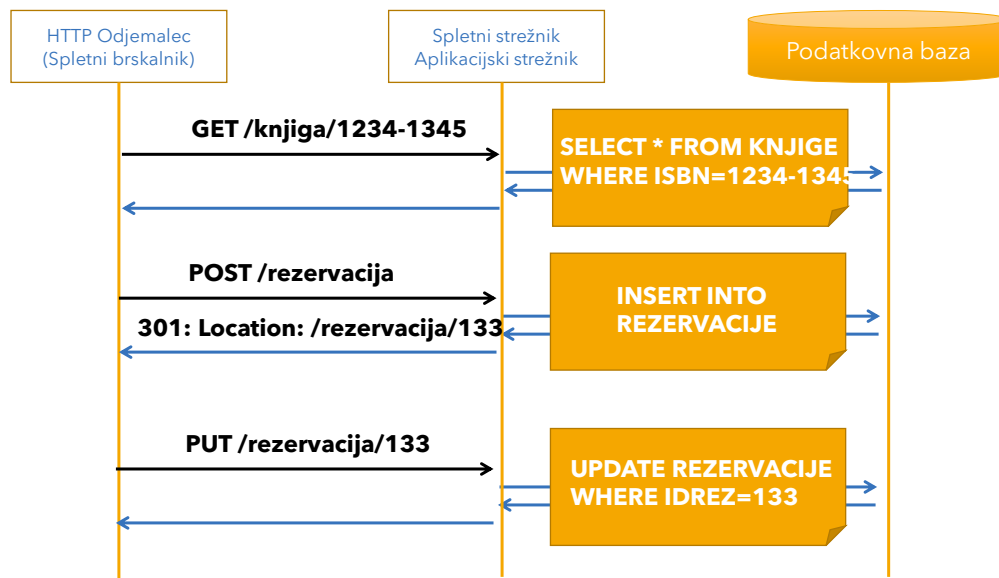
142

WS-* spletna storitev



143

RESTful spletna storitev



144

WS-* ali REST

- Obeh pristopov ne moremo direktno primerjati!
- Katero alternativo izbrati?
 - odvisno od posameznega primera do posameznega primera oz. problema, ki ga naslavljamo
- Ni enostavnega odgovora!

145

WS-* ali REST

- Vsak izmed pristopov ima svoje prednosti - podpira določene arhitekturne principe, tehnologije
 - **Način integracije** (skupna baza, prenos datotek, RPC, sporočilni sistemi)
 - **Pristop k izgradnji pogodb** (contract-first, contract-last, brez pogodb)
 - **Identifikacija virov**
 - **Povezovanje virov**
 - **Vzorci izmenjave sporočil**
 - **Prenosni protokol** (HTTP, TCP, SMTP, JMS, ...)
 - **Format sporočil** (SOAP, XML, JSON, RSS, ...)
 - **Identifikacija sporočil** (URI, WS-Addressing)
 - **Opis storitev** (tekstovni opis, XML Schema, WSDL, WADL,...)
 - **Zanesljivost dostave** (HTTPR, WS-ReliableMessaging, ...)
 - **Varnost** (HTTPS, WS-Security, ...)
 - **Transakcije** (WS-AT, WS-BA,...)
 - **Kompozicija storitev** (BPEL, spletne sestavljanke,...)
 - **Odkrivanje storitev** (UDDI, lastne rešitve)
 - ...

146

Zagotavljanje varnosti (WS-*)

- XML Signature (2001)
- XML Encryption (2002)
- SOAP razširitve, ki jih definira WS-Security (2004)
- Varna komunikacija na celotni poti sporočila (točka-do-točke)
 - **zaščiteni SOAP sporočila ne zahtevajo prenosa po HTTPS**

147

Zagotavljanje varnosti (REST)

- Varnost je implementirana na osnovi HTTPS (HTTP + SSL/TLS)
- HTTP Basic Authentication, OAuth, OAuth2, OpenID, JWT, itd.
- Varna komunikacija
 - avtentikacija, integriteta in enkripcija

148

Asinhrono in/ali zanesljivo sporočanje (WS-*)

- SOAP sporočila se lahko prenašajo z uporabo asinhronih prenosnih protokolov in APIjev (JMS, MSMQ, ...)
- WS-Addressing se lahko uporabi za prenos sporočil do končnih točk, ki je neodvisen od prenosnega protokola
- WS-ReliableExchange in WS-ReliableMessaging definirata protokol za zanesljivo dostavo sporočil

149

Še nekaj misli glede primerjave WS-* in REST

- Dejstvo je, da je potrebno izbrati določeno tehnologijo in povezan sklad tehnologij, s katero bomo rešili nalogo oz. dosegli zahtevano funkcionalnost in lastnosti –
- **Ne obstaja pristop, ki je nabolj primeren za reševanje vseh nalog!**
- **WS-*** zagotavlja sklad tehnologij, ki omogočajo **močno ogrodje** za razvoj specifičnih rešitev, a hkrati lahko zavira razvoj drugih rešitev
 - Enako velja za REST
- Odločitev, katero tehnologijo izbrati, je tako popolnoma odvisna od zahtev aplikacije in omejitev

150

Ali lahko gradimo SOA z REST?

151

REST prihodnost za storitveno usmerjene arhitekture?

- V obstoječi literaturi je razširjeno mnenje, da je REST prihodnost SOAe
- V obstoječi literaturi lahko zasledimo tudi prispevke, v katerih avtorji primerjajo REST in SOAP
 - Nekaj vidikov smo primerjali tudi v prejšnjih prosojnicah
 - **Primerjava nikakor ni trivialna!**

152

REST prihodnost za SOA?

- V zadnjem času sta se razširila predvsem 2 pristopa:

- REST kot **tehnološki pristop k izgradnji spletnih storitev**
(REST storitev oz. HTTP APIjev)

in

- **PRAVI REST**

153

Pravi REST

- Je **učinkovita implementacija arhitekture, ki temelji na virih (ROA - Resource-Oriented Architecture)**
 - ne gre samo za izbiro podporne tehnologije za implementacijo komunikacije
- Torej, namesto, da se vprašamo, če pravi REST omogoča izgradnjo SOA-e

je pomembno, da ugotovimo

Ali ROA, ki jo zgradimo na osnovi REST, zadovolji vse cilje/principe SOA?

154

Ali lahko dosežemo SOA z REST?

- Na višjem nivoju DA, vendar ne popolnoma
- SOA temelji na funkcijski dekompoziciji arhitekture podjetja
 - zahteva izgradnjo sistema/rešitve na osnovi
 - **komponent** (diskretnih koščkov kode in/ali podatkovnih struktur)
 - **poslovnih storitev** (poslovnih funkcij)
 - **poslovnih procesov** (orkestracija procesov, v splošnem na osnovi BPEL)
- ROA temelji na dekompoziciji arhitekture na množico virov
 - Vsak vir je direktno-dostopna porazdeljena komponenta, do katere pridemo preko standardnega/enotnega vmesnika

155

WS-* : REST

"If WS- is the RPC of the Internet, REST is the DBMS of the internet"*

- SOA integracija omogoča komunikacijo različnih programskih komponent skozi metode storitev
- REST omogoča izgradnjo komponent oz. virov, ki navzven ponudijo obnašanje tabel in operacij za delo s tabelami
 - **INSERT, SELECT, UPDATE, DELETE,**
 - oz. če preslikamo v HTTP: **POST, GET, PUT, DELETE**

156

Storitev : Vir

Storitev	Vir
<i>Samo-vsebovana, neodvisno razvita, nameščena, upravljana in vzdrževana programska komponenta, ki podpira specifično poslovno funkcionalnost.</i>	<i>Direktno dosegljiva, neodvisno razvita, nameščena, upravljana in vzdrževana programska komponenta, ki omogoča določene podatke. Vir lahko vsebuje tudi povezave na druge vire.</i>

157

Vmesniki

Storitev	Vir
<i>Funkcionalnost storitve je definirana v javno dostopnem vmesniku (na primer WSDL)</i>	<i>Vir je definiran z njegovim URL in vsebino vhodov/izhodov. Akcije so vsebovane v predstavitvi vira (princip HATEOAS)</i>

158

Ali lahko dosežemo SOA z REST?

- SOA temelji na principih
 - **Standardizacija pogodb storitev**
 - **Šibka sklopljenost storitev**
 - **Abstrakcija storitev**
 - **Ponovna uporaba storitev**
 - **Avtonomnost storitev**
 - **Storitve brez stanja**
 - **Sposobnost odkrivanja**
 - **Sposobnost kompozicije**
- Ali lahko s storitvami REST zagotovimo principe?

159

Ali lahko dosežemo SOA z REST?

- **SOA arhitektura zahteva storitve z jasno definiranimi vmesniki** (service contracts).
- REST ponuja arhitekturni slog za implementacijo takih storitev, ne določa pa enotnega standarda za opis vmesnika storitev. Na voljo je več različnih pristopov in specifikacij, npr.:
 - WADL,
 - WSDL 2.0 (omejena uporaba pri REST),
 - novejša specifikacije, kot je npr. OpenAPI (Swagger).
- V primerjavi s SOAP/WSDL na področju RESTful storitev ni enega univerzalno sprejetega standarda, ki bi pokril vse vidike opisa storitev na način, kot to počne WSDL v svetu SOAP.
- Sisteme, ki temeljijo na REST, je vsekakor mogoče zasnovati v skladu z načeli SOA, vendar to ni samoumevno:
 - zahteva disciplinirano modeliranje storitev in virov,
 - dosledno upravljanje API-jev (governance, verzioniranje),
 - ter dodatne pristope za formalni opis vmesnikov.

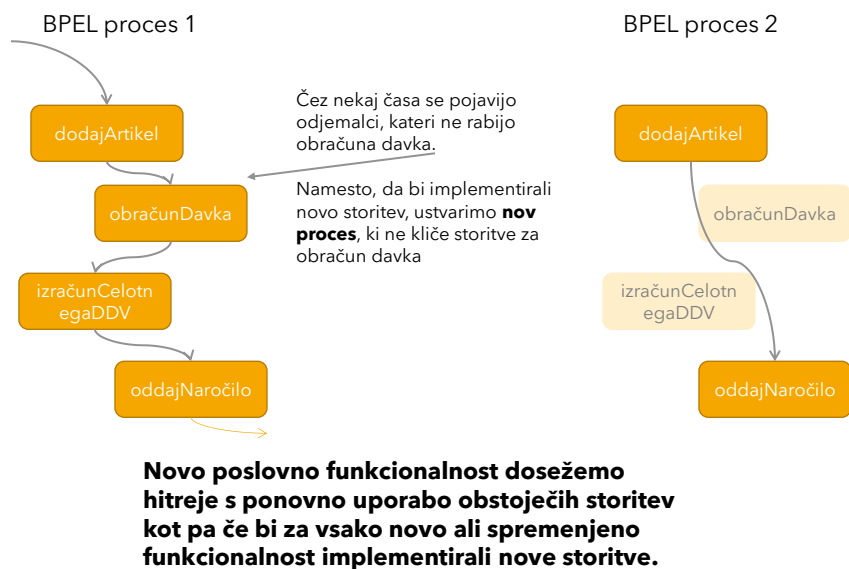
160

Pomen orkestracije za SOA

- **Sposobnost orkestracije je glavna prednost SOAe**
 - Orkestracija je potrebna za kompozicijo storitev v poslovne procese
- **Uporaba BPEL ali podobnega mehanizma (ESB, usmerjanje,...) je ključna za SOAo!**

161

Pomen orkestracije za SOA



162

Kompozicija storitev (WS-*)

- WS-BPEL je standardni jezik za kompozicijo spletnih storitev.
- Modeli poslovnih procesov v jeziku WS-BPEL definirajo kako se orkestrira množica spletnih storitev v kompozicijo storitev

163

Kompozicija storitev (REST)

- Osnovni principi REST ne vključujejo koncepta kompozicije storitev
 - Je pa v okviru REST nastal koncept sestavljanja spletnih virov za razvoj **spletnih sestavljanjk** (MashUps)
- Glavno vprašanje je:
- **Ali lahko WS-BPEL uporabimo tudi za RESTful storitve?**

164

WS-BPEL in WSDL 2.0

WSDL 2.0 HTTP Binding lahko ovije RESTful spletne storitve. Vendar, iz specifikacij WS-BPEL 2.0:

At the time this specification was completed, various Web Service standards work, such as WSDL 2.0 and WS-Addressing, were ongoing and not ready for consideration for WS-BPEL 2.0. Future versions of WS-BPEL may provide support for these standards.

